# Defect-Aware Multi-Die Image Fusion for Semiconductor Reference Reconstruction

Research and implementation documentation

Md Shamiul Islam Riyad · B.Sc. CSE, United International University

October 2026 · Version 1.0 · Planning document: no experiments have been run

# Contents

**How to read this document**

**A. Executive summary and research motivation**
- A.1 Problem
- A.2 Proposed research
- A.3 Intended application and potential benefit
- A.4 Hypotheses to be tested
- A.5 Major technical risks
- A.6 Recommended strategy

**B. Problem formulation**
- B.1 Scenarios that must not be confused
- B.2 Notation
- B.3 Observation model
- B.4 Registration transformations
- B.5 Defect masks, reliability and the three states
- B.6 Fusion function and outputs
- B.7 What the formulation implies

**C. Literature review and research gap**
- C.1 Semiconductor wafer and die defect inspection
- C.2 Industrial anomaly detection and segmentation
- C.3 Image registration and geometric alignment
- C.4 Robust multi-image fusion
- C.5 Masked image reconstruction and inpainting
- C.6 Uncertainty estimation and hallucination control
- C.7 Established methods versus the proposed contribution
- C.8 Research gap: stated narrowly

**D. Dataset investigation and data strategy**
- D.1 Summary
- D.2 Public datasets investigated
- D.3 Data plan: three levels
- D.4 Synthetic benchmark design (Level 2)
- D.5 Annotation requirements (Level 3 and any real masks)
- D.6 Splits and leakage prevention

**E. Proposed system architecture**
- E.1 Stage specifications

**F. Model selection and technical justification**
- F.1 Registration
- F.2 Defect evidence
- F.3 Fusion and reconstruction
- F.4 Staged strategy

**G. Detailed algorithm, equations and pseudocode**
- G.1 Registration to a common frame
- G.2 Defect probabilities from each evidence source
- G.3 Reliability weights and support
- G.4 Robust, defect-aware fusion
- G.5 Pixels without reliable evidence, uncertainty and abstention
- G.6 Running reconstruction only when appropriate
- G.7 Training losses

**H. Hardware and software requirements**
- H.1 What drives memory and time
- H.2 Configurations
- H.3 Recommendations by task
- H.4 Software stack and reproducible environment

**I. Training and experimentation protocol**
- I.1 Data preprocessing and annotation format
- I.2 Synthetic defect generation and controlled transformations
- I.3 Objectives and losses
- I.4 Hyper-parameter starting points (initial values, not optima)
- I.5 Checkpoints, seeds, logging and tracking
- I.6 Validation, early stopping and profiling
- I.7 Leakage controls

**J. Evaluation framework**
- J.1 Evaluation sets
- J.2 Metrics
- J.3 Methods compared
- J.4 Fair-comparison protocol
- J.5 Ablation studies
- J.6 What cannot be validated without clean references or real same-design data

**K. Risk analysis and limitations**

**L. Implementation roadmap**

**M. Expected outputs**

**N. References and appendices**
- References
- N.1 Verification log
- N.2 Dataset cards (summary)
- N.3 Installation
- N.4 Configuration example
- N.5 Glossary
- N.6 Optional animation storyboard (about 60 seconds)

# How to read this document

This document plans a research project. It does not report results. No model has been trained, no dataset has been processed, and no hardware has been benchmarked. Every number in it is one of three things: a property of a published dataset or paper, with a link; a stated starting value for an experiment; or a calculation shown in full.

Key statements carry one of four labels:

| Label | Meaning |
|---|---|
| **Fact** | Supported by a cited source or by a calculation shown here. |
| **Recommendation** | A design choice made for this project, with the reason given. It can be changed. |
| **Hypothesis** | A claim the project will test. It may turn out to be false. |
| **Assumption** | A condition the method needs. If it fails, the method's guarantees fail with it. |

**Figures.** Figures 1–6 are schematics drawn from code (`make_figures.py`). None shows real inspection data. Figure 3 is a conceptual example and is labelled as one. No third-party images are reproduced, because none of the reviewed datasets clearly permits redistribution in a document of this kind.

**Verification.** The build environment could not open publisher and dataset websites directly. Bibliographic details were checked through search results that quote those pages. Appendix N.1 lists which details were confirmed this way and which still need a manual check.

# A. Executive summary and research motivation

## A.1 Problem

Automated inspection of semiconductor dies often compares an image of a die with a **reference image** of what the die should look like. The reference can be a neighbouring die (die-to-die comparison), a rendering of the design (die-to-database), or a stored "golden" image [11, 15]. If the reference is itself a real inspection image, any particle, scratch, bridge, missing feature, noise or imaging artefact in it enters the comparison. The result is missed defects or false alarms.

**Fact.** Commercial inspection already derives references from several dies, for example a per-pixel median or mean over about five to eight dies [15] or a computed reference fitted to a set of dies [16]. The same patent literature notes a known failure: a median cannot serve as the reference when systematic defects are widespread, because the median itself is then defective [17].

## A.2 Proposed research

This project studies how to build a **candidate defect-free reference image** from 4–5 images of dies that share one intended design. Each image may carry defects in different places. The system should:

1. align the images into one coordinate frame;
2. estimate, for every image and region, whether it is reliable, defective or uncertain;
3. compare corresponding regions across images;
4. fuse only trustworthy evidence;
5. reconstruct the reference where enough evidence exists;
6. explicitly flag regions where all observations are defective or the evidence is insufficient.

**The output is a digital image plus maps:** a candidate reference, per-image defect and reliability masks, an uncertainty map, an abstention mask and an evaluation report. It is not a fabricated or repaired die. A clean-looking reconstruction says nothing about electrical function.

## A.3 Intended application and potential benefit

- **Cleaner references for comparison-based inspection.** A reference that excludes defects seen in individual dies could reduce false alarms and missed detections compared with using one neighbouring image (Hypothesis H1).
- **Visible reliability.** An explicit uncertainty or abstention map tells an engineer where the reference can be trusted, instead of hiding gaps behind plausible-looking pixels.
- **A reproducible open benchmark.** A synthetic same-design benchmark with exact ground truth would let methods be compared fairly. To our knowledge no public dataset supports this today (Section D).

## A.4 Hypotheses to be tested

| ID | Hypothesis | Main measurement (Section J) |
|---|---|---|
| H1 | Defect-aware fusion leaks fewer defects into the reference than unweighted median or mean fusion, at comparable structural fidelity, when defects are mostly random. | Defect leakage rate, edge F-score |
| H2 | Support-based abstention flags regions that cannot be recovered (for example, defects shared by all dies) with high recall. | Abstention recall on the unrecoverable set $Z$ |
| H3 | Reliability that combines learned defect scores with cross-die consistency beats either source alone. | Leakage and PSNR ablation |
| H4 | Residual registration error is a major driver of reconstruction error. The threshold at which it dominates is unknown and will be measured. | Error versus injected registration error |
| H5 | Detectors trained or calibrated on synthetic and public data transfer only partly to real same-design SEM groups. The gap can be quantified only with real data. | Synthetic versus real metrics |

## A.5 Major technical risks

1. **Repeated defects.** A systematic or mask-level defect at the same location in every die cannot be recovered by fusion. It can only be flagged, and it passes into the reference if it goes undetected.
2. **Registration error.** Misalignment blurs edges or mixes structures. Periodic layouts can cause shifts of exactly one pattern pitch.

3. **False-positive defect masks.** Normal process variation flagged as a defect removes good evidence.
4. **Hallucination.** Learned reconstruction can invent plausible but wrong circuitry.
5. **Data.** No public dataset supplies same-design die groups with verified clean references. Real validation depends on industry or laboratory collaboration, and synthetic results may not transfer.
6. **Prior art.** Multi-die reference generation already exists in industry [15–17]. The research contribution must be stated narrowly (Section C.8).

### A.6 Recommended strategy

**Recommendation.** Proceed in stages:

1. **Stage 1:** classical registration, robust statistics and mask-weighted fusion with abstention, built and evaluated first.
2. **Stage 2:** learned defect segmentation and reliability estimation (U-Net, PatchCore or PaDiM).
3. **Stage 3:** a learned reconstruction model, only for unresolved regions and only if it passes hallucination checks.

Diffusion models are optional, pretrained only, and never presented as verified patterns.

## B. Problem formulation

### B.1 Scenarios that must not be confused

| Scenario | What the images are | Addressed here? |
|---|---|---|
| **S1. Same-design dies, complementary defects** | Images of *different physical dies* printed from the same design (same layer, same process step), taken with the same tool settings. Random defects differ between dies. | **Yes. This is the target scenario.** |
| S2. Multiple views of one physical die | Repeated images of the *same* die (re-scans, frame averaging, multiple detectors). A defect appears in every view. | No. Fusion would reproduce the defect. S2 helps only with noise, not defects. |
| S3. Different lighting or imaging conditions | Same or different dies under different illumination, detector, beam energy or focus. | Only as a nuisance variable inside S1 (Section B.6). Strong changes break the shared-appearance assumption. |
| S4. Wafer maps | Grids of categorical labels per die (for example pass/fail or failure-pattern class), as in WM-811K [8] and MixedWM38 [9]. | No. Wafer maps contain no die-surface images, so nothing can be reconstructed from them. |

**Assumptions for S1.** The method is only valid under the following conditions:

- **A1:** all dies share one intended layout in the region of interest (ROI).
- **A2:** the images come from the same layer and process step, with the same nominal magnification and detector.
- **A3:** at most locations, the majority of dies are defect-free; defects are not systematically co-located.
- **A4:** geometric differences between images can be modelled by a low-dimensional transform plus small residual distortion.
- **A5:** appearance differences between dies are mostly a global gain and offset plus noise.

When A3 fails locally (repeated defects), the correct output is abstention, not reconstruction.

### B.2 Notation

| Symbol | Plain-English meaning |
|---|---|
| $N$ | Number of images in a group (planned: 4–5; ablated: 2–5). |
| $k = 1, \dots, N$ | Index of an image (one die). |
| $\Omega \subset \mathbb{Z}^2$ | Pixel grid of the common reference frame (the ROI). |
| $\mathbf{x} \in \Omega$ | A pixel position in that frame. |
| $I_k$ | Raw image of die $k$, defined on its own grid $\Omega_k$. |
| $R(\mathbf{x})$ | The unknown clean reference: the intended pattern as this tool would image it. |
| $\mathbf{T}_k$ | Geometric transform mapping reference coordinates to image-$k$ coordinates. |
| $g_k$ | Photometric map for image $k$ (brightness and contrast). |
| $M_k(\mathbf{x}) \in \{0, 1\}$ | True defect mask of die $k$ (1 = defective). Unknown in real data. |

| Symbol | Plain-English meaning |
|---|---|
| $D_k(\mathbf{x})$ | Appearance of the defect where $M_k = 1$. |
| $\varepsilon_k(\mathbf{x})$ | Noise. |

## B.3 Observation model

$$I_k\big(\mathbf{T}_k(\mathbf{x})\big) \;=\; \big(1 - M_k(\mathbf{x})\big)\, g_k\big(R(\mathbf{x})\big) \;+\; M_k(\mathbf{x})\, D_k(\mathbf{x}) \;+\; \varepsilon_k(\mathbf{x}), \qquad \mathbf{x} \in \Omega.$$

In words, at every reference position die $k$ shows either the intended pattern (after a brightness and contrast change $g_k$) or a defect, plus noise. The baseline photometric model is affine, $g_k(r) = a_k r + b_k$ with gain $a_k > 0$ and offset $b_k$. **Assumption.** Charging, edge effects and detector non-linearity in SEM images can violate this model; that is tested as a nuisance factor.

## B.4 Registration transformations

Registration estimates $\hat{\mathbf{T}}_k$ for each image. The candidate families, from most to least constrained, are:

| Family | Form | Degrees of freedom | When to use |
|---|---|---|---|
| Translation | $\mathbf{T}(\mathbf{x}) = \mathbf{x} + \mathbf{t}$ | 2 | Stage-aligned images, same magnification |
| Euclidean (rigid) | $\mathbf{T}(\mathbf{x}) = \mathbf{R}_\theta \mathbf{x} + \mathbf{t}$ | 3 | Small rotations |
| Similarity | $\mathbf{T}(\mathbf{x}) = s\,\mathbf{R}_\theta \mathbf{x} + \mathbf{t}$ | 4 | Rotation plus magnification drift |
| Affine | $\mathbf{T}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{t}$ | 6 | Shear or anisotropic scale (e.g. scan distortion) |
| Residual field | $\mathbf{T}(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{t} + \mathbf{u}(\mathbf{x})$ | dense | Small non-rigid distortion (optional, regularised) |

Here $\mathbf{R}_\theta$ is a 2-D rotation by angle $\theta$, $s$ is a scale factor, $\mathbf{A}$ is a 2×2 matrix and $\mathbf{u}$ is a smooth displacement field.

**Recommendation.** Use the least flexible family that explains the data. Extra freedom lets the registration "explain away" real defects.

The **registered image** is $\tilde{I}_k(\mathbf{x}) = I_k(\hat{\mathbf{T}}_k(\mathbf{x}))$, computed by bilinear or bicubic interpolation. A validity mask $V_k(\mathbf{x}) \in \{0, 1\}$ marks pixels whose source position lies inside image $k$.

**Photometric correction** puts all images on a common scale:

$$y_k(\mathbf{x}) = \big(\tilde{I}_k(\mathbf{x}) - \hat{b}_k\big)/\hat{a}_k,$$

where $\hat{a}_k, \hat{b}_k$ are fitted robustly (for example by least trimmed squares against the group median), so that defects do not bias the fit.

## B.5 Defect masks, reliability and the three states

For every image and pixel the system estimates the following quantities:

- **Defect probability** $\hat{p}_k(\mathbf{x}) \in [0, 1]$: from a segmentation network, or from a calibrated anomaly score (Section G.2).
- **Reliability weight** $w_k(\mathbf{x}) \in [0, 1]$: how much observation $y_k(\mathbf{x})$ should count in fusion. It combines defect probability, cross-die consistency, registration confidence and validity (Section G.3).
- **State**, for display and counting only:
  - *defective* if $\hat{p}_k(\mathbf{x}) \geq \tau_d$;
  - *reliable* if $w_k(\mathbf{x}) \geq \tau_r$ and not defective;
  - *uncertain* otherwise.

**These are estimates.** In real data the true mask $M_k$ is unknown, so a high anomaly score is evidence of a defect, not a confirmed defect.

**Support** is the number of reliable observations at a pixel:

$$S(\mathbf{x}) = \sum_{k=1}^{N} \mathbb{1}\big[w_k(\mathbf{x}) \geq \tau_r\big].$$

## B.6 Fusion function and outputs

A **fusion function** $F$ maps the stack of observations and weights to an estimate:

$$\hat{R}(\mathbf{x}) = F\big(\{y_k(\mathbf{x}), w_k(\mathbf{x})\}_{k=1}^{N}\big), \quad \text{defined only where } \mathbf{x} \notin A.$$

The **abstention mask** $A(\mathbf{x}) \in \{0, 1\}$ marks pixels where no trustworthy estimate exists, for example $S(\mathbf{x}) < s_{\min}$. At those pixels $\hat{R}$ is stored as "not a number" (NaN) and is never silently filled.

The **uncertainty map** $U(\mathbf{x}) \geq 0$ summarises how unreliable $\hat{R}(\mathbf{x})$ is, from the support and the spread of the reliable observations (Section G.5).

The deliverables per group are:

- $\hat{R}$;
- $\{\hat{p}_k, w_k\}$;
- $S$, $U$ and $A$;
- a **provenance map** recording which dies contributed to each pixel;
- an evaluation record.

### B.7 What the formulation implies

- **Fact (from the model).** Where every die is defective at $\mathbf{x}$, no observation carries information about $R(\mathbf{x})$, so the reference there is unidentifiable from the images alone. Only external information (a verified golden die or design data) can resolve it.
- **Fact (robust statistics).** The pixel-wise median has a 50% breakdown point. If half or more of the observations at a pixel are defective, the median can be arbitrarily wrong. With $N = 4$, two coinciding defects already reach that limit.
- **Hypothesis.** Learned defect evidence can identify defective observations even when they are the majority at a pixel, which pure cross-die statistics cannot. This is tested in the overlap ablation (Section J.5).

## C. Literature review and research gap

**Scope.** This review covers the six areas the project builds on. It is a starting point, not the systematic review that milestone L1 requires, which must also search patents and SPIE/JM3 inspection literature in a documented way. Each table summarises the objective and method, the data used, strengths and limitations, and relevance to this project. Dataset papers are covered in Section D.

### C.1 Semiconductor wafer and die defect inspection

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| Dehaerne et al., systematic review [11] | Systematic review of SEM-based automatic defect inspection; 103 papers screened from four databases (Jan 2024). | Published SEM inspection studies | Broad, recent; reports that reference-based detection was the most common approach until 2020, after which deep learning dominated, especially for classification. Review only. | Confirms the place of reference-based methods and the shift to learning; entry point for L1. |
| Zontak & Cohen [14] | Defect detection in patterned wafers *without registration*: anisotropic-kernel reconstruction of the source image from a reference image; features that cannot be reconstructed from the reference are flagged. | Patterned-wafer SEM images | Handles misalignment implicitly; reference-based. Assumes the reference itself is clean. | Closest academic precedent for "reconstruct from a reference". This project instead reconstructs the *reference* from several imperfect dies. |
| Dey et al. [12] | Instance segmentation (Mask R-CNN) of SEM defects: bridges, breaks, line collapses; per-defect masks for counting and area. | SEM images of patterned resist structures | Pixel-level masks on real SEM; supervised, needs annotations. | Candidate learned defect-evidence model (Stage 2). |
| De Ridder et al., SEMI-DiffusionInst [13] | Diffusion-based detection and instance segmentation of SEM defects. | SEM defect images | Reports mAP gains over the authors' earlier work; heavier inference. | Shows diffusion used for *segmentation*, not reconstruction; a Stage-2 alternative if compute allows. |
| Taha, survey [10] | Taxonomy and experimental comparison of ML methods for wafer-defect classification. | Wafer maps | Broad comparison; classification of maps, not die images. | Background: classification is well studied. Image-level labels are not enough here (Section F.2). |

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| KLA-Tencor patents [15–17] | Standard reference die from mean/median of about 5–8 dies [15]; computed reference minimising difference-image norm [16]; median unusable when systematic defects are widespread, use a verified template [17]. | Industrial wafer inspection | Show multi-die references are established industrial practice. Patents disclose claims, not reproducible benchmarks. | **Directly limits the novelty claim** (Section C.8). |

## C.2 Industrial anomaly detection and segmentation

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| PatchCore, Roth et al. [18] | One-class anomaly detection and localisation: memory bank of pretrained CNN patch features from defect-free images, reduced by greedy coreset subsampling; nearest-neighbour distance gives a per-patch anomaly score. | MVTec AD and two further datasets | Strong localisation, no defect labels needed. Memory bank grows with data; scores are uncalibrated distances; backbone is pretrained on natural images. | Stage-2 defect evidence when only defect-free images exist. |
| PaDiM, Defard et al. [19] | Gaussian model of pretrained-CNN patch embeddings at each position; Mahalanobis distance as anomaly score. | MVTec AD, STC | Simple and fast; position-specific models suit *aligned* images. Needs good alignment; covariance per position costs memory. | Well matched to registered same-design images; Stage-2 baseline. |
| DRAEM, Zavrtanik et al. [20] | Reconstructive plus discriminative sub-networks trained on synthetic anomalies pasted onto normal images. | MVTec AD | Learns segmentation without real defect labels. Synthetic anomaly realism limits transfer. | Template for synthetic defect generation (Section D.6) and a learned reconstruction idea. |
| Anomalib, Akcay et al. [21] | Open-source library implementing PatchCore, PaDiM and other methods with benchmarking tools. | Several public datasets | Reproducible reference implementations. Library defaults are not tuned for SEM. | Recommended implementation source for Stage-2 baselines. |
| U-Net, Ronneberger et al. [22] | Encoder–decoder with skip connections for dense segmentation from few annotated images. | Biomedical microscopy | Strong with small datasets; needs pixel labels. | Stage-2 supervised segmentation and Stage-3 masked reconstruction backbone. |
| MVTec AD (extended), Bergmann et al. [2] | Dataset paper, plus the per-region-overlap (PRO) metric for localisation. | MVTec AD | Standard benchmark and metric. Industrial objects, not dies. | AU-PRO for segmentation evaluation (Section J.2). |

## C.3 Image registration and geometric alignment

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| Zitová & Flusser [23] | Survey: area-based versus feature-based registration; four steps (features, matching, mapping model, resampling). | Survey | Standard taxonomy. Predates learned registration. | Structures the registration design (Section E, stage 3). |
| Reddy & Chatterji [28] | FFT-based registration of translation, rotation and scale (Fourier–Mellin, log-polar). | General images | Global, fast, no features needed. Sensitive to large non-overlapping content. | Coarse alignment when rotation or scale is unknown. |
| Guizar-Sicairos et al. [29] | Efficient sub-pixel translation registration by matrix-multiply DFT upsampling. | Phase-retrieval images | Accurate to a fraction of a pixel at low cost; translation only. Basis of scikit-image `phase_cross_correlation`. | Stage-1 translation registration and initialisation. |
| Evangelidis & Psarakis, ECC [27] | Iterative parametric alignment maximising the enhanced correlation coefficient; invariant to global brightness and contrast. | General images | Sub-pixel, robust to gain/offset; needs a reasonable initialisation; local optimum. Implemented as OpenCV `findTransformECC`. | Stage-1 refinement for Euclidean/affine models. |
| Lowe, SIFT [24]; Rublee et al., ORB [25] | Keypoint detection and description (SIFT: scale-invariant gradients; ORB: fast binary features). | General images | Handles large rotation or scale. Repetitive layouts produce ambiguous matches. | Coarse alignment when transforms are large and features are distinctive. |
| Fischler & Bolles, RANSAC [26] | Robust model fitting from matches containing outliers. | General | Standard outlier rejection. Fails when most matches are wrong (periodic patterns). | Pairs with SIFT/ORB. |

## C.4 Robust multi-image fusion

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| Candès et al., Robust PCA [30] | Decompose a data matrix into low-rank plus sparse parts (Principal Component Pursuit) with exact-recovery guarantees under conditions. | Video surveillance, faces | Principled separation of a shared component from sparse corruptions. Needs many observations; $N$ = 4–5 is very few. | Conceptual model: aligned dies ≈ shared pattern + sparse defects. |
| Peng et al., RASL [31] | Jointly solves for alignment transforms and a low-rank plus sparse decomposition of a batch of images. | Faces, digits, video frames | Unifies registration and robust fusion. Convex relaxation per iteration; assumes linearly correlated images and moderate misalignment. | **Strong baseline** for joint alignment and defect separation; compare with the staged pipeline. |
| Farsiu et al. [32] | Multi-frame super-resolution with L1 data term and robust regularisation, giving median-like fusion that resists outliers. | Video sequences | Shows the robustness of L1/median estimators to outliers and misregistration. Targets resolution, not defects. | Justifies L1/weighted-median fusion (Section G.4). |

## C.5 Masked image reconstruction and inpainting

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| Telea [33] | Fast-marching inpainting propagating smooth information into holes. | General images | Simple, deterministic; blurs structure in large holes. In OpenCV. | Visual comparison only; must never be shown as recovered structure. |
| Liu et al., partial convolutions [34] | Convolutions conditioned only on valid pixels, with automatic mask update. | Natural-image benchmarks | Handles irregular masks. Learns plausible fills, not verified ones. | Masked U-Net building block for Stage 3. |
| Suvorov et al., LaMa [35] | Large-mask inpainting with fast Fourier convolutions for a global receptive field. | Natural-image benchmarks | Good at repetitive, periodic structure, which is relevant to layouts. Can hallucinate convincingly. | Candidate pretrained comparison for Stage 3, strictly flagged. |
| Lugmayr et al., RePaint [36] | Inpainting with a pretrained unconditional diffusion model, changing only the reverse sampling. | Faces and general images | Any mask shape. Stochastic; slow; outputs are samples, not estimates. | Only an optional experiment; samples could feed an uncertainty estimate. |
| He et al., masked autoencoders [37] | Self-supervised pre-training by reconstructing masked patches. | ImageNet | Learns structure from unlabelled data. Reconstructions are blurry and not meant as outputs. | Possible self-supervised pretraining on unlabelled SEM images. |

## C.6 Uncertainty estimation and hallucination control

| Work | Objective and method | Data | Strengths / limitations | Relevance |
|---|---|---|---|---|
| Kendall & Gal [38] | Separates aleatoric (data) and epistemic (model) uncertainty in dense prediction; heteroscedastic loss. | Segmentation and depth | Per-pixel uncertainty in one framework. Heteroscedastic variance can be miscalibrated. | Loss for the Stage-3 model (Section G.7). |
| Lakshminarayanan et al. [39]; Gal & Ghahramani [40] | Deep ensembles; Monte Carlo dropout as approximate Bayesian inference. | Various | Practical epistemic uncertainty. Ensembles cost $M$× compute; MC dropout often underestimates. | Uncertainty for learned stages. |
| Geifman & El-Yaniv [41] | Selective classification: abstain to guarantee risk at a chosen coverage. | Image classification | Formalises risk-coverage trade-offs. | Selective-risk and AURC metrics (Section J.2). |
| Guo et al. [42] | Modern networks are often miscalibrated; temperature scaling fixes much of it. | Image classification | Simple post-hoc calibration. | Calibrating defect probabilities $\hat{p}_k$. |
| Ilg et al. [43] | Uncertainty for optical flow; sparsification curves and AUSE. | Sintel, KITTI | Dense-uncertainty evaluation without calibration assumptions. | AUSE metric (Section J.2). |
| Bhadra et al. [44] | Defines and maps hallucinations in learned image reconstruction (measurement versus null-space components). | Stylised tomography | Shows learned reconstructions can add false structure. | Motivates abstention by default and hallucination checks (Section K). |

## C.7 Established methods versus the proposed contribution

**Established (not claimed).**

- Die-to-die and multi-die median references [15–17].
- Reference-based detection [11, 14].

- Supervised and one-class defect segmentation [12, 18–22].
- Translation, rigid, affine and feature-based registration [23–29].
- Low-rank plus sparse and robust fusion [30–32].
- Learned inpainting [33–37].
- Uncertainty and selective prediction [38–44].

### C.8 Research gap: stated narrowly

**What may be open.** This is a hypothesis to confirm in the systematic review. Defect *detection* and median references exist. What we have not found is a published, reproducible study that combines and evaluates all of the following:

1. same-design multi-die registration;
2. *per-image, region-level* defect and reliability estimates combining learned evidence with cross-die consistency;
3. evidence-weighted reconstruction with recorded provenance;
4. *explicit abstention and uncertainty* for regions without sufficient evidence;
5. an open benchmark with exact ground truth that measures defect leakage, structural fidelity and abstention quality.

**What would remove the gap.** Any prior work found in L1 that already does items 2–5 together. If found, the project should become a reproduction and comparison study, or narrow its focus to the benchmark or to abstention.

**Recommendation.** Present the work as a test of whether this combination helps, never as a new kind of inspection.

## D. Dataset investigation and data strategy

### D.1 Summary

**Fact.** None of the investigated public datasets contains groups of same-design die images with complementary defects *and* a clean reference of the same die. None therefore supports direct evaluation of registration plus reconstruction against ground truth.

The public data is useful for two things:

1. **training and calibrating defect evidence** (segmentation and anomaly scoring);
2. **supplying realistic defect appearances** for a synthetic benchmark.

| | Semicon-ductor | Patterned layout | Pixel masks | Same-design groups | Clean ref. of same die | Registration ground truth | Reconstruction ground truth | Licence clear |
|---|---|---|---|---|---|---|---|---|
| **Carinthia [4]** SEM, one layer | yes | no | no | no | no | no | no | unclear |
| **Carinthia-S [5]** SEM + masks | yes | no | yes | no | no | no | no | partial |
| **MVTec AD [1, 2]** industrial objects | no | partial | yes | no | no | no | no | partial |
| **MVTec AD 2 [3]** industrial, lighting | no | partial | partial | no | no | no | no | partial |
| **KolektorSDD2 [6]** surface defects | no | no | yes | no | no | no | no | partial |
| **VisA, PCB subsets [7]** same-design PCBs | no | yes | yes | partial | no | no | no | unclear |
| **WM-811K [8]** wafer maps | yes | no | no | no | no | no | no | unclear |
| **MixedWM38 [9]** wafer maps | yes | no | no | no | no | no | no | unclear |
| **Synthetic benchmark** planned (Level 2) | partial | yes | yes | yes | yes | yes | yes | yes |
| **Real same-design groups** target (Level 3) | yes | yes | partial | yes | partial | partial | partial | partial |

Rows above the dashed line are public datasets as described by their providers; rows below are planned data. “Partial” and “unclear” are explained in Section D. No public dataset found provides same-design groups with complementary defects and a clean reference of the same die.

Figure 1. Dataset suitability for each requirement of the task. Public datasets above the dashed line are rated from their providers’ descriptions; rows below are the planned data levels. Schematic table, not experimental data.

## D.2 Public datasets investigated

Sources: providers' pages and papers [1–9], read through search-result quotations (see Appendix N.1). "Third-party" marks details found only in secondary listings.

| Dataset | What it contains | Patterned dies? | Same-design groups / clean ref.? | Direct evaluation of alignment or reconstruction? | Access and licence | Suitability |
|---|---|---|---|---|---|---|
| **Carinthia** [4] (Zenodo 10715190, v2, 2024) | 4,591 grayscale SEM images of defects from one production layer; 6 classes; CSV with image path and label (image-level labels). | No: described as *unstructured* wafers. | No / No | No | Open download from Zenodo. Licence **not verified**; record also reported under another DOI (concept vs. version). | Real SEM defect appearance; classification pre-training. |
| **Carinthia-S** [5] (Zenodo 16895427, 2025) | Same 4,591 images with binary segmentation masks; CSV maps image, mask and label. Third-party: 480×480 px, class 6 = "no visible defect" incl. misaligned captures, some masks anti-aliased. | No | No / No | No | Open download; **CC BY 4.0 according to third-party listings** (confirm on record). | Best public source of *real SEM defect masks* for cut-and-paste synthesis and U-Net training. |
| **MVTec AD** [1, 2] | 5,354 high-resolution colour images, 15 object and texture categories, >70 defect types; defect-free training set; pixel-precise anomaly masks. | No (industrial objects/textures) | No / No (defect-free images are other instances) | No | Official download; **CC BY-NC-SA 4.0**, non-commercial. | Anomaly-detection baselines; defect masks for synthesis; not die imagery. |
| **MVTec AD 2** [3] | >8,000 high-resolution images, 8 scenarios incl. lighting changes, transparent and overlapping objects, very small defects; public test set with masks plus a private test set (ground truth withheld, evaluation server). | No | No / No | No | Official download; **CC BY-NC-SA 4.0**. | Robustness to lighting; small-defect behaviour. |
| **KolektorSDD2** [6] | 3,335 images (356 with visible defects, 2,979 defect-free), about 230×630 px, from production items captured in a controlled industrial setting; pixel-level defect annotations. | No (surface images) | No / No | No | Download from ViCoS; **CC BY-NC-SA 4.0** per the ViCoS page (a mirror lists CC BY-SA, so use the ViCoS terms); commercial use by agreement. | Mixed and weak supervision; surface-defect segmentation. |
| **VisA**, PCB subsets [7] | 10,821 images (9,621 normal, 1,200 anomalous) in 12 subsets, 4 of them printed circuit boards; pixel masks. | **Partly**: PCB boards of one design are patterned, but not semiconductor dies. | Partial: many instances of the same PCB design; not paired, not aligned / No | Partly, as a proxy: groups of normal boards can be registered, but the boards are physically different instances. | GitHub release; **licence conflicting** (CC BY 4.0 in the repository vs CC BY-NC-SA 4.0 in Anomalib docs). | Best public *patterned same-design proxy*; useful to test registration on real repeated layouts. |
| **WM-811K** [8] | 811,457 wafer maps (die-level categorical grids); a labelled subset with failure-pattern classes (secondary sources: about 172,950 labelled). | No (maps, not images) | No / No | No | Widely mirrored; licence unclear. | Not usable for reconstruction (scenario S4). |
| **MixedWM38** [9] | More than 38,000 wafer maps with 1 normal, 8 single and 29 mixed defect patterns. | No (maps) | No / No | No | IEEE DataPort; licence unclear. | Not usable for reconstruction (S4). |

**Other semiconductor literature.** The SEM studies [11–13] use proprietary or partner data; most are not public. The systematic review [11] should be used in L1 to search for any public patterned-die SEM data.

## D.3 Data plan: three levels

**Level 1: public data for defect evidence (pre-training and baselines).**

- Carinthia-S masks → supervised U-Net defect segmentation on real SEM texture (unpatterned background).
- MVTec AD, MVTec AD 2, KolektorSDD2 → PatchCore/PaDiM baselines and segmentation pre-training.
- VisA PCB → registration and fusion sanity checks on real repeated layouts.

None of these is used as reconstruction ground truth.

**Level 2: controlled synthetic benchmark (main quantitative evaluation).** Groups are generated from known clean references, so $\boldsymbol{R}$, $\boldsymbol{M_k}$, $\mathbf{T}_k$ and the unrecoverable set $\boldsymbol{Z}$ are known exactly (Section D.4).

**Level 3: real same-design die groups (external validity).** Real SEM or optical images of 4–5 dies of the same design and layer, ideally with a verified clean reference: a die confirmed defect-free by review, a design-rendered image, or a high-dose rescan. These are obtained through a university fab, laboratory or industry partner under a data agreement. **Without Level 3, the project cannot claim real-world validity** (Section J.6).

## D.4 Synthetic benchmark design (Level 2)

**Clean references.** Three sources are planned:

- a. procedurally generated layouts (lines, vias, contacts, pads) rendered with an SEM-like appearance model: edge brightening, blur, shot-like noise;
- b. defect-free crops of real patterned images where licensing allows (e.g. VisA PCB normal images, *only* as a proxy);
- c. Level-3 verified references when available.

**Group generation** (all parameters drawn from seeded distributions and logged):

1. Choose a clean reference $\boldsymbol{R}$ and $\boldsymbol{N} \in \{2, \ldots, 5\}$.
2. For each die $\boldsymbol{k}$, sample a transform $\mathbf{T}_k$:
   - translation up to a few pixels to tens of pixels;
   - rotation within ±θ_max;
   - scale within $\mathbf{1} \pm \boldsymbol{s}_{\max}$;
   - optional smooth distortion $\mathbf{u}$.
3. Sample a photometric map $\boldsymbol{g_k}$ (gain, offset, slow illumination gradient) and noise $\boldsymbol{\varepsilon_k}$ (Gaussian plus Poisson-like; optional blur).
4. Sample defects: type, size, count and location, under a **controlled overlap** policy:
   - *independent* (random positions);
   - *pairwise overlap* at a set rate;
   - *all-N* (the same location in every die, simulating systematic or mask-level defects).
5. Insert defects. Defect appearances are cut from Carinthia-S or MVTec masks and blended (alpha or Poisson blending), or drawn procedurally (bridge, open, particle, scratch, residue).
6. Render $\boldsymbol{I_k}$ by applying $\boldsymbol{g_k}$, $\mathbf{T}_k^{-1}$, defects and noise. Store $\boldsymbol{R}$, $\boldsymbol{M_k}$, $\mathbf{T}_k$, all parameters and the derived $\boldsymbol{Z} = \{\mathbf{x} : \sum_k (1 - M_k(\mathbf{x})) < s_{\min}\}$.

**Realism controls.**

- Compare intensity histograms, noise power spectra and edge profiles of synthetic images with real SEM images (Carinthia, and Level 3 when available).
- Report every metric separately for procedural versus cut-and-paste defects.
- **Synthetic results are never reported as real-world performance.**

## D.5 Annotation requirements (Level 3 and any real masks)

| Item | Requirement |
|---|---|
| Defect masks | Polygon or pixel masks per die; class label; annotator ID; confidence (certain / probable / unsure). |
| Clean reference | Source (reviewed die, design rendering, re-scan); reviewer; date; known caveats. |
| Correspondence | Die IDs, wafer, lot, layer, tool, recipe, magnification, timestamp. |
| Registration truth | Optional fiducial points (≥ 4 per image) for target-registration error on real data. |
| Format | PNG/TIFF images (16-bit if available), PNG masks, one JSON or CSV manifest per group with SHA-256 hashes. |

**Quality control.**

- Double annotation of at least a random subset, reporting inter-annotator IoU.
- Adjudication of disagreements.
- An “unsure” class that is excluded from precision and recall but reported separately.
- Periodic audit of 5–10% of masks.

## D.6 Splits and leakage prevention

**Recommendation.** Split by **design or reference identity and acquisition group**, never by individual image.

- **Synthetic data.** Each clean reference (layout) belongs to exactly one split: train, validation or test. All groups generated from one reference stay in that split. The test split also holds out at least one *layout family* never seen in training, to measure generalisation.
- **Real data.** Split by wafer or lot and, where possible, by layer or design. Never place dies from the same wafer region in both train and test. Near-duplicate images (re-scans, overlapping fields of view) are clustered by perceptual hash and kept in one split.
- **Public datasets.** Keep providers’ official train/test splits; never tune thresholds on test images.
- **Thresholds and hyper-parameters** ($\tau_d$, $\tau_r$, $s_{\min}$, $\kappa$) are chosen on validation groups only, then frozen before test evaluation.
- **Manifest hashing.** Every split is defined by a versioned manifest of file hashes; the evaluation script refuses to run if a test hash appears in the training manifest.

# E. Proposed system architecture

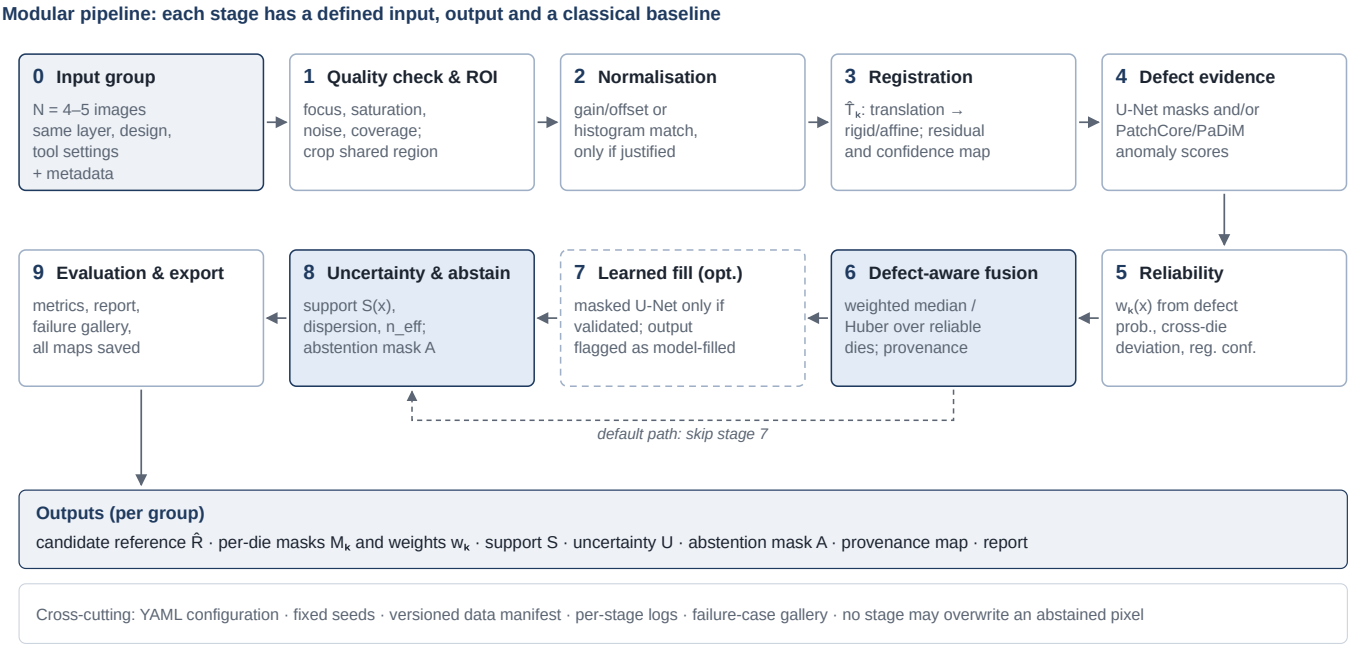



Figure 2. Modular pipeline. Stages 6 and 8 are the core of the proposed method; stage 7 is optional and skipped by default. Schematic, not experimental data.

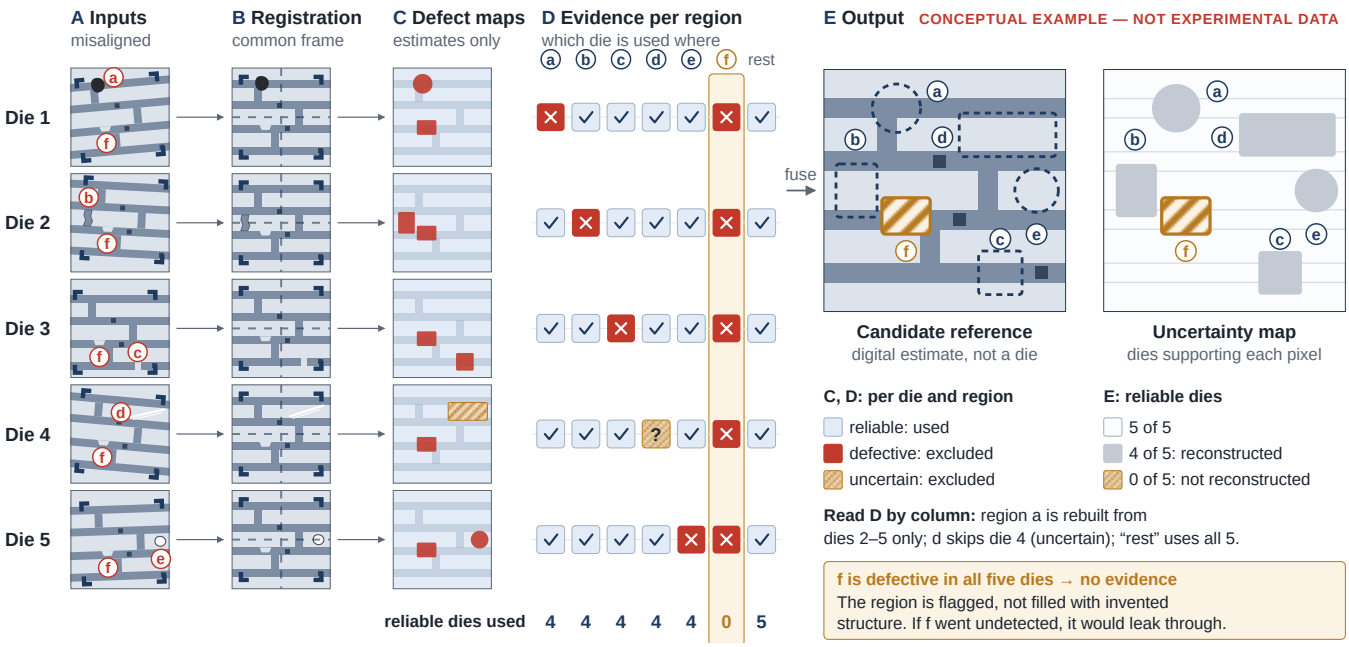



Figure 3. Conceptual example (synthetic, not experimental data). One layout is used throughout. (A) Five misaligned dies, each with its own defect a–e, and defect f repeated in all five. (B) Registration into one frame. (C) Estimated defect maps. (D) Which die supplies each region, and how many reliable dies support it. (E) Candidate reference and uncertainty map; f has no reliable source and is flagged, not filled.

## E.1 Stage specifications

Each stage below lists its inputs, outputs, candidate methods, assumptions, failure modes and the **recommended baseline** to build first.

**Stage 1. Image quality checks and ROI extraction.**

- *Inputs:* raw images $\boldsymbol{I_k}$ and metadata.
- *Outputs:* accepted images, ROI, quality report.
- *Methods:*
  - focus measure (variance of Laplacian);
  - saturation fraction;
  - noise estimate (median absolute deviation of a high-pass residual);
  - coverage of the shared ROI after coarse alignment.
- *Assumptions:* metadata identifies layer and design.
- *Failure modes:* thresholds rejecting usable images; charging artefacts passing.
- *Baseline:* fixed thresholds set on validation data; reject an image only with a logged reason; require at least 3 accepted images.

**Stage 2. Illumination and contrast normalisation (only when justified).**

- *Inputs:* accepted images.
- *Outputs:* $\boldsymbol{y_k}$ on a common intensity scale, plus fitted $\hat{\boldsymbol{a}}_{\boldsymbol{k}}, \hat{\boldsymbol{b}}_{\boldsymbol{k}}$.
- *Methods:*
  - robust affine gain/offset to the group median (least trimmed squares);
  - histogram matching;
  - low-order background (illumination gradient) removal.
- *Assumptions:* A5 (mostly global photometric differences).
- *Failure modes:* histogram matching can make a large defect look normal or vice versa, or create false structure.
- *Baseline:* robust gain/offset only. Use histogram matching only if validation shows the gain/offset model leaves systematic residuals. **Normalisation is fitted after coarse registration**, using only pixels with low cross-die deviation.

**Stage 3. Geometric registration.**

- *Inputs:* images, ROI.
- *Outputs:* $\hat{\mathbf{T}}_k$, registered $\tilde{I}_k$, validity $V_k$, residual and confidence maps $c_k$.
- *Methods:* phase correlation [29]; Fourier–Mellin [28]; ECC [27]; SIFT/ORB + RANSAC [24–26]; optional regularised dense residual flow; joint alignment by RASL [31] as a comparison.
- *Assumptions:* A4; enough non-periodic structure, or fiducials, to resolve pitch ambiguity.
- *Failure modes:*
  - one-pitch shifts in periodic arrays;
  - defects pulling the alignment;
  - over-flexible models absorbing defects;
  - interpolation blur.
- *Baseline:* the reference frame is the median-quality image (or the design rendering if available). Phase correlation for translation, then ECC Euclidean refinement, with defect-likely pixels masked out of the ECC objective in a second pass.

**Stage 4. Defect segmentation and anomaly scoring.**

- *Inputs:* registered, normalised images.
- *Outputs:* $\hat{p}_k(\mathbf{x})$, defect probability per image.
- *Methods:* supervised U-Net [22] (Mask R-CNN [12] if instance counts matter); one-class PatchCore [18] / PaDiM [19]; cross-die robust deviation.
- *Assumptions:* training domain resembles test domain (H5).
- *Failure modes:*
  - false positives on process variation;
  - false negatives on low-contrast defects;
  - miscalibrated scores.
- *Baseline:* cross-die robust z-score (no learning). Learned models are added in Stage 2 of the strategy, with temperature or isotonic calibration on validation data [42].

**Stage 5. Reliability and confidence estimation.**

- *Inputs:* $\hat{p}_k$, deviation $z_k$, registration confidence $c_k$, validity $V_k$.
- *Outputs:* weights $w_k(\mathbf{x})$, states (reliable / defective / uncertain), support $S(\mathbf{x})$.
- *Methods:* product-of-evidence weight (Equation G.3); optionally a small learned fusion of the evidence channels.
- *Assumptions:* evidence sources are not all wrong at once.
- *Failure modes:* with two coincident defects at $N = 4$ the median breaks down and the consistency term points the wrong way.
- *Baseline:* the product weight with thresholds tuned on validation groups.

**Stage 6. Defect-aware fusion.**

- *Inputs:* $y_k$, $w_k$.
- *Outputs:* $\hat{R}$, provenance map.
- *Methods:* weighted median; Huber M-estimator via iteratively reweighted least squares; region-level source selection where reliable observations disagree structurally.
- *Assumptions:* reliable observations show the same structure up to noise.
- *Failure modes:* edge blur from residual misregistration; seams between regions from different sources; mixing of distinct structures (Section G.4).
- *Baseline:* weighted median over observations with $w_k \geq \tau_r$.

**Stage 7. Optional learned reconstruction.**

- *Inputs:* stack of $y_k$, $w_k$, and the abstention mask $A$.
- *Outputs:* a fill for some abstained pixels, the model-filled mask $F$, and predicted uncertainty.
- *Methods:* masked U-Net with partial convolutions [34]; pretrained LaMa [35]; RePaint [36] as an experiment only.
- *Assumptions:* the layout is locally predictable from context (often false for unique structures).
- *Failure modes:* hallucinated circuitry [44].

- *Baseline:* **none: skipped by default.** Enabled only after it passes the checks in Section K.

**Stage 8. Uncertainty estimation and abstention.**

- *Inputs:* $S$, the weighted spread, $n_{\text{eff}}$, the region consistency flag and $F$.
- *Outputs:* $U$, $A$.
- *Methods:* analytic spread and support (Section G.5); ensembles or MC dropout for learned stages [39, 40].
- *Assumptions:* calibration on validation groups transfers to test.
- *Failure modes:* overconfidence when every reliable die shares an undetected defect (unobservable from the images).
- *Baseline:* $A = [S < s_{\min}] \vee [\text{SE} > \tau_u]$.

**Stage 9. Evaluation and result export.**

- *Inputs:* all maps, ground truth where available.
- *Outputs:* metrics (Section J), report, failure gallery, files.
- *Format:* 32-bit float TIFF for $\hat{R}$ (NaN where abstained); PNG/TIFF masks; JSON metrics; YAML copy of the configuration; provenance as a compact index map.

# F. Model selection and technical justification

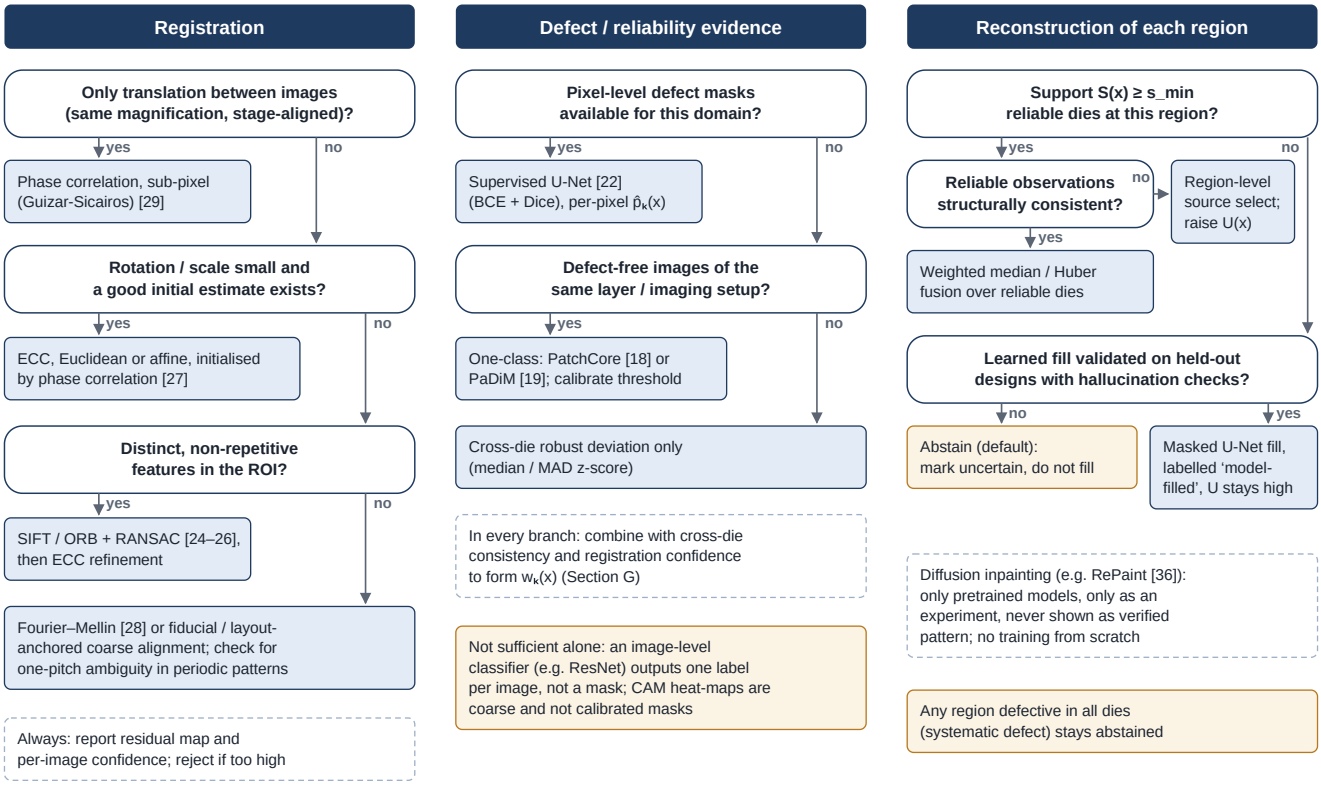



Figure 4. Decision diagram for choosing registration, defect-evidence and reconstruction methods. Recommendations, not results.

## F.1 Registration

| Option | Handles | Pros | Cons | Recommendation |
|---|---|---|---|---|
| Phase correlation (sub-pixel) [29] | Translation | Fast, accurate, global, intensity-robust | Translation only; periodic ambiguity | **Default first step** |
| Fourier–Mellin [28] | Translation, rotation, scale | No features needed | Less accurate; boundary effects | Coarse alignment if rotation is unknown |
| ECC [27] (OpenCV) | Translation to homography | Sub-pixel; gain/offset invariant; supports masks | Needs initialisation; local optimum | **Default refinement** (Euclidean, then affine if needed) |
| SIFT / ORB + RANSAC [24–26] | Large rotation and scale | Wide capture range | Repetitive layouts give wrong matches; ORB less precise | Only with distinctive structures or fiducials |

| Option | Handles | Pros | Cons | Recommendation |
|---|---|---|---|---|
| Learned registration | Anything learned | Can handle complex distortion | Needs training data; opaque failure | Not recommended for this data scale |
| RASL [31] | Joint align and separate | Uses all images together | Heavier; assumptions on correlation | **Comparison baseline** |

## F.2 Defect evidence

| Option | Supervision | Output | Pros | Cons | Recommendation |
|---|---|---|---|---|---|
| Cross-die robust z-score | None | Per-pixel deviation | No training; uses the group itself | Breaks at ≥ 50% co-located defects | **Stage-1 baseline** |
| U-Net [22] | Pixel masks | Per-pixel probability | Precise; data-efficient | Needs masks (Carinthia-S, synthetic) | **Stage-2 main** |
| PatchCore [18] | Defect-free only | Patch anomaly score (upsampled) | No defect labels; strong localisation | Uncalibrated; memory bank grows | **Stage-2 alternative** |
| PaDiM [19] | Defect-free only | Position-wise Mahalanobis score | Uses alignment; simple | Per-position covariance memory | Stage-2 alternative for aligned data |
| DRAEM [20] | Synthetic anomalies | Mask | No real labels | Synthetic-to-real gap | Optional |
| ResNet classifier | Image labels | One label per image | Simple | **No pixel mask, no reconstruction** | Not sufficient (below) |

**Why a classification-only model cannot do this task.** A classifier such as a ResNet maps an entire image to one label (“bridge”, “particle”, “clean”). Fusion needs to know *which pixels* of *which image* are untrustworthy, and reconstruction needs pixel values. Class activation maps from a classifier are coarse (at the resolution of the last feature map), uncalibrated and not trained to delineate defect boundaries. They cannot replace a segmentation mask. A classifier can still serve as a quality gate (e.g. rejecting a badly charged image) or route groups to defect-specific settings.

## F.3 Fusion and reconstruction

| Option | Pros | Cons | Recommendation |
|---|---|---|---|
| Pixel-wise median (unweighted) | Simple; robust below 50% outliers | Ignores defect evidence; blurs edges under mis-registration; mixes structures; fails at ≥ 50% | Baseline M2 |
| Trimmed mean | Lower variance than median | Same failure modes | Baseline M2 variant |
| Weighted median / Huber (proposed) | Uses evidence; robust; provenance is traceable | Depends on weight quality | **Stage-1 core (M3)** |
| Region-level source selection | Keeps edges sharp; no mixing | Seams; depends on region definition | Used where structure disagrees |
| Masked U-Net / partial conv [34] | Fills gaps from context | Can hallucinate; needs training data | Stage 3, optional, flagged |
| LaMa [35] (pretrained) | Strong on periodic texture | Domain gap; convincing hallucinations | Comparison only |
| Diffusion inpainting [36] | Diverse samples, usable for uncertainty | Slow; stochastic; data-hungry | **Pretrained only; never trained from scratch here** |

**Why not train a large diffusion model from scratch.** The expected data (a few thousand SEM images at most, plus synthetic data) is far too small to train a large generative model well. Training cost is high. More importantly, a generative model is designed to produce *plausible* content, which is the opposite of what an inspection reference needs where evidence is missing. If used at all, pretrained diffusion inpainting is an experiment whose outputs are labelled as samples, and whose spread across samples may serve as an uncertainty signal.

## F.4 Staged strategy

1. **Stage 1: classical.** Phase correlation + ECC registration; robust gain/offset; cross-die z-score; weighted-median fusion; support-based abstention. Deliverable: a complete, reproducible baseline pipeline (M2 and M3 in Section J).
2. **Stage 2: learned evidence.** U-Net on Carinthia-S and synthetic masks; PatchCore/PaDiM on defect-free data; calibrated probabilities feeding $w_k$. Deliverable: ablation H3.

3. **Stage 3: optional reconstruction.** Masked U-Net for abstained regions, only if hallucination checks pass on held-out designs. Outputs stay flagged (M4).

# G. Detailed algorithm, equations and pseudocode

All thresholds below are **initial values to be tuned on validation groups**. None is a proven optimum.

## G.1 Registration to a common frame

```
procedure REGISTER_GROUP(images I[1..N], roi, model ∈ {translation, euclidean, affine})
    ref ← argmax_k quality(I[k])                # or a design rendering, if available
    for k in 1..N:
        # 1. coarse: translation by sub-pixel phase correlation on windowed, high-passed images
        t0[k] ← phase_cross_correlation(hp(I[ref]), hp(I[k]), upsample=20)
        if model ≠ translation and rotation/scale unknown:
            (θ0, s0) ← fourier_mellin(I[ref], I[k])
        # 2. refine: ECC with the chosen motion model, initialised from step 1
        T[k] ← findTransformECC(I[ref], I[k], init=(t0, θ0, s0), model, mask=roi)
    # 3. second pass: recompute after masking likely defects (pixels with high cross-die deviation)
    D ← robust_deviation_mask(warp_all(I, T))
    for k in 1..N:
        T[k] ← findTransformECC(I[ref], I[k], init=T[k], model, mask=roi ∧ ¬D[k])
    # 4. diagnostics
    for k in 1..N:
        Ĩ[k], V[k] ← warp(I[k], T[k]), valid_mask(T[k])
        ρ[k] ← 1 − local_ncc(Ĩ[k], median_k(Ĩ), window=w)   # local residual
        c[k] ← exp(−ρ[k]² / (2 σρ²))
        if pitch_ambiguity(I[ref], T[k]) or mean(ρ[k]) > ρ_max: flag image k (exclude or review)
    return T, Ĩ, V, c
```

Notes:

- `hp` is a high-pass filter that suppresses illumination gradients.
- `pitch_ambiguity` tests whether shifting by one layout period gives a near-equal correlation peak. If so, the group needs fiducials or a design-anchored reference.
- *Initial values:* upsample factor 20 (1/20 px), window $w$ = 15 px, σ_ρ = 0.2.

## G.2 Defect probabilities from each evidence source

**Cross-die robust deviation (no learning).** With registered, photometrically corrected observations $y_k(\mathbf{x})$:

$$m(\mathbf{x}) = \underset{k:\, V_k(\mathbf{x})=1}{\operatorname{median}} y_k(\mathbf{x}), \qquad s(\mathbf{x}) = 1.4826 \underset{k}{\operatorname{median}} |y_k(\mathbf{x}) - m(\mathbf{x})|,$$

$$z_k(\mathbf{x}) = \frac{|y_k(\mathbf{x}) - m(\mathbf{x})|}{\max\left(s(\mathbf{x}),\, s_0\right)}.$$

What the symbols mean:

- $m$ is the per-pixel median over the dies.
- $s$ is the median absolute deviation (MAD). The factor 1.4826 makes it estimate a standard deviation for Gaussian noise.
- $s_0$ is a floor equal to the global noise level, so that flat regions (where $s \approx 0$) do not inflate $z_k$.
- $z_k$ says how many noise-units die $k$ is away from the consensus.

**Recommendation.** Compute $z_k$ on lightly smoothed images, or as a patch-level statistic, so that sub-pixel misregistration at edges is not mistaken for a defect.

**Learned evidence.** $\hat{p}_k^{\text{seg}}(\mathbf{x})$ from a U-Net, or an anomaly score $a_k(\mathbf{x})$ from PatchCore/PaDiM mapped to a probability by a calibration function fitted on validation data:

$$\hat{p}_k^{\text{anom}}(\mathbf{x}) = \sigma\big((a_k(\mathbf{x}) - \mu_a)/\tau_a\big),$$

where $\sigma$ is the logistic function and $\mu_a, \tau_a$ are fitted. Isotonic regression is an alternative.

**Combined defect probability.** A noisy-OR keeps the most alarming source:

$$\hat{p}_k(\mathbf{x}) = 1 - \big(1 - \hat{p}_k^{\text{learn}}(\mathbf{x})\big)\big(1 - \hat{p}_k^{\text{cons}}(\mathbf{x})\big), \qquad \hat{p}_k^{\text{cons}}(\mathbf{x}) = 1 - \exp\!\big(-z_k(\mathbf{x})^2 / 2\kappa^2\big),$$

with $\kappa$ (initial value 3) controlling how many noise-units count as “inconsistent”. Small mask post-processing follows: morphological opening to remove isolated pixels, and dilation by *r* px (initial 2) so that defect borders are excluded.

## G.3 Reliability weights and support

$$w_k(\mathbf{x}) = V_k(\mathbf{x}) \cdot \big(1 - \hat{p}_k(\mathbf{x})\big) \cdot c_k(\mathbf{x}) \cdot q_k,$$

- $V_k$ is image validity (0 outside the warped image);
- $1 - \hat{p}_k$ is the probability that the pixel is *not* defective;
- $c_k$ is local registration confidence (Section G.1);
- $q_k \in [0, 1]$ is a per-image quality score from stage 1 (initial: 1 for accepted images).

Reliability states: defective if $\hat{p}_k \geq \tau_d$ (initial 0.5); reliable if $w_k \geq \tau_r$ (initial 0.5) and not defective; otherwise uncertain. Support $S(\mathbf{x})$ is the count of reliable observations (Section B.5).

## G.4 Robust, defect-aware fusion

Let $\mathcal{K}(\mathbf{x}) = \{k : w_k(\mathbf{x}) \geq \tau_r\}$ be the reliable dies at $\mathbf{x}$.

**Weighted median (default).**

$$\hat{R}(\mathbf{x}) = \arg\min_{r} \sum_{k \in \mathcal{K}(\mathbf{x})} w_k(\mathbf{x}) \; |y_k(\mathbf{x}) - r|.$$

Computed by sorting the $y_k$ and taking the value where the cumulative weight first reaches half the total weight.

**Huber M-estimator (variant).** Solved by iteratively reweighted least squares starting from the weighted median:

$$\hat{R}(\mathbf{x}) = \arg\min_{r} \sum_{k \in \mathcal{K}(\mathbf{x})} w_k(\mathbf{x})\, \rho_\delta\big(y_k(\mathbf{x}) - r\big), \qquad \rho_\delta(u) = \begin{cases} \tfrac{1}{2}u^2 & |u| \leq \delta \\ \delta\,(|u| - \tfrac{1}{2}\delta) & |u| > \delta \end{cases}$$

with $\delta = 1.345\, s_0$ as an initial value. That is the classical tuning constant for 95% efficiency under Gaussian noise.

**Weighted mean** (only for comparison):

$$\hat{R}(\mathbf{x}) = \frac{\sum_{k \in \mathcal{K}} w_k y_k}{\sum_{k \in \mathcal{K}} w_k}.$$

**Why the pixel-wise median is not universally valid.**

1. *Residual misregistration.* If die $k$ is shifted by $\delta_k$, a sharp edge appears at different positions. The pixel-wise median places the edge near the median shift only when a majority agree. With $N = 4$ the median averages the two middle values, which blurs edges, and a mean blurs further.
2. *Mixing of distinct structures.* If reliable dies show genuinely different local structures (e.g. a one-pitch misregistration, or a design variant), choosing the median independently per pixel can stitch pieces of different structures into a pattern that exists in no die.
3. *Breakdown.* At 50% or more defective observations the median follows the defects.
4. *Photometric seams.* With imperfect gain correction, the median may switch source die from pixel to pixel, adding texture-like noise.

**Structure-aware safeguard.** Partition $\Omega$ into regions $r$: tiles of 16–32 px, or superpixels of $m(\mathbf{x})$. For each region, measure structural disagreement among reliable dies, $E_r$ (e.g. mean pairwise edge-map distance). If $E_r > \tau_e$:

- select one source $k_r^\star = \arg\max_k \frac{1}{|r|} \sum_{\mathbf{x} \in r} w_k(\mathbf{x})$;
- copy $y_{k_r^\star}$ into the region with feathered blending at the boundary;
- record the source in the provenance map;
- raise $U$ in that region.

```
procedure FUSE(y[1..N], w[1..N], regions, τr, s_min)
    for each pixel x:
        K ← {k : w[k](x) ≥ τr}
        S(x) ← |K|
        if S(x) ≥ s_min:  R̂(x) ← weighted_median({y[k](x)}_{k∈K}, {w[k](x)}_{k∈K})
        else:             R̂(x) ← NaN                        # handled in G.5, never filled here
        provenance(x) ← bitmask(K)
    for each region r with S ≥ s_min on most pixels:
        if structural_disagreement(r, K) > τe:
            k* ← argmax_k mean_{x∈r} w[k](x)
            R̂|r ← feather_blend(R̂|r, y[k*]|r);  provenance|r ← {k*};  flag_region(r)
    return R̂, S, provenance, flagged_regions
```

## G.5 Pixels without reliable evidence, uncertainty and abstention

**Effective sample size and weighted spread.**

$$n_{\text{eff}}(\mathbf{x}) = \frac{\left(\sum_{k\in\mathcal{K}} w_k\right)^2}{\sum_{k\in\mathcal{K}} w_k^2}, \qquad \hat{\sigma}^2(\mathbf{x}) = \frac{\sum_{k\in\mathcal{K}} w_k \left(y_k - \hat{R}\right)^2}{\sum_{k\in\mathcal{K}} w_k}.$$

$n_{\text{eff}}$ is how many "full" observations the weights amount to. It equals $|\mathcal{K}|$ when all weights are equal and is smaller when one die dominates. $\hat{\sigma}^2$ is the spread of the reliable observations around the estimate.

**Approximate standard error.** This is exact for a weighted mean with independent noise. For a median it is approximate and should be checked by bootstrap on validation groups.

$$\mathrm{SE}(\mathbf{x}) = \sqrt{\frac{\max\left(\hat{\sigma}^2(\mathbf{x}),\, s_0^2\right)}{n_{\text{eff}}(\mathbf{x})}}.$$

**Uncertainty map.** $U = \mathrm{SE}/s_0$ (in noise units) for supported pixels; $U = +\infty$ (displayed as "unsupported") where $S < s_{\min}$; and $U \leftarrow \max(U, u_{\text{flag}})$ in structurally flagged regions.

**Abstention mask.**

$$A(\mathbf{x}) = \mathbb{1}\left[S(\mathbf{x}) < s_{\min}\right] \vee \mathbb{1}\left[\mathrm{SE}(\mathbf{x}) > \tau_u\right] \vee \mathbb{1}\left[\mathbf{x} \in \text{flagged region with no single reliable source}\right].$$

Initial values are $s_{\min} = 2$ and $\tau_u = 2\,s_0$:

- $s_{\min} = 1$ would accept a single unchecked observation;
- $s_{\min} = 2$ requires two dies to agree;
- the ablation tests 1, 2 and 3.

```
procedure ABSTAIN_AND_UNCERTAINTY(R̂, y, w, S, flagged, s_min, τu)
    for each pixel x:
        if S(x) < s_min:            A(x) ← 1; U(x) ← ∞; R̂(x) ← NaN; continue
        n_eff ← (Σ w)² / Σ w²;  σ̂² ← Σ w (y − R̂(x))² / Σ w
        SE ← sqrt(max(σ̂², s0²) / n_eff);  U(x) ← SE / s0
        if flagged(x):              U(x) ← max(U(x), u_flag)
        A(x) ← (SE > τu)
        if A(x): R̂(x) ← NaN
    return U, A                     # A is exported unchanged; no later stage may clear it silently
```

## G.6 Running reconstruction only when appropriate

```
procedure OPTIONAL_FILL(R̂, A, y, w, model, validated: bool)
    if not validated: return R̂, F = 0                    # default: leave NaN, keep A
    candidates ← {x : A(x) = 1 and S(x) ≥ 1 and hole_size(x) ≤ h_max}
    if candidates = ∅: return R̂, F = 0
    R̃, b ← model(stack(y, w), mask = A)                  # prediction and predicted scale b
    for x in candidates:
        R̂(x) ← R̃(x);  F(x) ← 1;  U(x) ← max(U(x), b(x)/s0, u_model)   # stays high
    # Systematic-defect pixels (S(x) = 0) are never filled, so A(x) stays 1
    return R̂, F
```

The model is "validated" only if, on held-out *designs*, it meets every one of these conditions:

- it beats leaving the hole empty on the agreed error metric;
- its hallucination rate (Section K) is below a pre-set limit agreed with the supervisor;
- its predicted scale $b$ is calibrated: AUSE better than random ordering.

### G.7 Training losses

**Segmentation (U-Net).** Binary cross-entropy plus Dice, with predicted probability $p$ and ground truth $g$ over pixels $i$:

$$\mathcal{L}_{\text{seg}} = -\frac{1}{|\Omega|}\sum_i \left[g_i \log p_i + (1-g_i)\log(1-p_i)\right] \; + \; \left(1 - \frac{2\sum_i p_i g_i + \epsilon}{\sum_i p_i + \sum_i g_i + \epsilon}\right),$$

with $\epsilon = 1$ for stability. Dice counteracts the small fraction of defect pixels.

**Masked reconstruction (optional Stage 3).** The network predicts $\tilde{R}$ and a per-pixel Laplace scale $b > 0$, giving a heteroscedastic loss [38], plus gradient and SSIM terms over training pixels $\Omega_t$ (holes weighted by $\lambda_h$):

$$\mathcal{L}_{\text{rec}} = \frac{1}{|\Omega_t|}\sum_{\mathbf{x}\in\Omega_t} \omega(\mathbf{x})\left[\frac{|\tilde{R}(\mathbf{x}) - R(\mathbf{x})|}{b(\mathbf{x})} + \log b(\mathbf{x})\right] + \lambda_g \left\|\nabla\tilde{R} - \nabla R\right\|_1 + \lambda_s\left(1 - \text{SSIM}(\tilde{R}, R)\right),$$

with $\omega(\mathbf{x}) = \lambda_h$ inside simulated holes and 1 elsewhere. *Initial values:* $\lambda_h = 4$, $\lambda_g = 0.5$, $\lambda_s = 0.2$. The first term penalises error relative to the predicted scale and stops the network from shrinking $b$ everywhere. The gradient term protects edges. SSIM protects local structure [45].

## H. Hardware and software requirements

**All hardware figures in this section are estimates for planning. None was measured.** Measure memory and runtime with a short profiling run (Section I.6) before buying or booking hardware.

| 1 CPU-only prototype<br>Classical registration, fusion, small experiments | 2 Entry-level GPU<br>Segmentation and modest reconstruction models | 3 Research workstation<br>High resolution, ablations, optional diffusion |
|---|---|---|
| CPU: 4–8 modern cores | CPU: 8–16 cores | CPU: 16–32 cores |
| RAM: 16 GB (32 GB for large images) | RAM: 32 GB | RAM: 64–128 GB |
| GPU: none | GPU: 8–12 GB VRAM (NVIDIA) | GPU: 16–24 GB, or 40–48 GB+ |
| Storage: SSD, 100–250 GB free | Storage: NVMe, 1 TB | Storage: NVMe 2–4 TB + backup |
| OS: Linux, Windows or macOS | OS: Ubuntu LTS; NVIDIA driver | OS: Ubuntu LTS; NVIDIA driver |
| **Typical use**<br>■ Inference on 4–5 images: yes<br>■ U-Net training: toy scale only<br>■ PatchCore: small images, slow<br>■ Diffusion: no | **Typical use**<br>■ U-Net on 256–512 px patches<br>■ PatchCore / PaDiM feature extraction<br>■ Masked U-Net reconstruction<br>■ Diffusion: pretrained inference, small | **Typical use**<br>■ Larger tiles, larger batches<br>■ Parallel ablation sweeps<br>■ Ensembles for uncertainty<br>■ Pretrained diffusion fine-tuning |
| *Estimates, not measured minimums* | *Estimates, not measured minimums* | *Estimates, not measured minimums* |

Figure 5. Three hardware configurations (estimates, not measured minimums).

### H.1 What drives memory and time

**Image arrays (calculation).** One float32 image of $H \times W$ pixels needs $4HW$ bytes:

- 4096×4096: 4 × 16,777,216 B ≈ 64 MiB.
- A group of 5 images keeps about 15 such arrays in memory (images, registered images, probabilities, weights, maps): about 1 GiB.
- At 8192×8192, each array is 256 MiB and 15 arrays are about 3.75 GiB. Tiling is then advisable.

**CNN training memory.** Activation memory grows roughly linearly with batch size × patch height × patch width × the summed channel widths of the network. Doubling the patch side therefore about quadruples activation memory. Mixed precision (fp16/bf16) roughly halves activation memory, and gradient checkpointing trades compute for memory. Model parameters

and optimiser state (Adam keeps two extra copies) add about 4× the parameter memory. This is small for a U-Net (tens of millions of parameters ≈ hundreds of MB).

**PatchCore memory bank (calculation).** Memory = images × patches per image × feature dimension × 4 bytes. *Example:* 200 training images, a 28×28 feature grid (784 patches) and 1,024-dimensional features give 200 × 784 × 1,024 × 4 B ≈ 642 MB before subsampling. A 10% coreset gives about 64 MB, and 1% about 6.4 MB. The nearest-neighbour search cost scales with bank size.

**Dataset size.** This affects storage and epoch time, not peak memory, provided data is streamed from disk.

## H.2 Configurations

| | 1. CPU-only prototype | 2. Entry-level GPU development | 3. Research workstation |
|---|---|---|---|
| Purpose | Preprocessing, classical registration, fusion, small experiments | Supervised segmentation, PatchCore/PaDiM, modest reconstruction | High resolution, large sweeps, ensembles, optional diffusion |
| CPU | Modern x86-64 or ARM64, 4–8 cores | 8–16 cores | 16–32 cores |
| System RAM | 16 GB (32 GB for > 4k images) | 32 GB | 64–128 GB |
| GPU VRAM | None | **8–12 GB** (NVIDIA, CUDA-capable) | 16–24 GB; 40–48 GB+ for diffusion fine-tuning or very large tiles |
| Storage | SSD, 100–250 GB free | NVMe SSD, 1 TB | NVMe SSD 2–4 TB plus backup/NAS |
| OS | Linux, Windows or macOS | Ubuntu LTS recommended; Windows/WSL2 possible | Ubuntu LTS |
| CUDA | Not needed | NVIDIA driver compatible with the chosen PyTorch CUDA build | Same |
| Workload limits | Learned models only at toy scale; PatchCore feasible but slow on small images; no diffusion | Patch-based training (256–512 px); pretrained inpainting inference at moderate resolution; no large-scale generative training | Diffusion fine-tuning only of pretrained models; still no training from scratch |

**CUDA note.** PyTorch's official pip and conda builds bundle the CUDA runtime libraries, so the main system requirement is an NVIDIA driver new enough for the CUDA version of the chosen build. Select the build with the selector at pytorch.org [48] at project start, record it in the lock file, and do not mix system CUDA toolkits with bundled ones unless compiling custom extensions.

## H.3 Recommendations by task

| Task | Recommendation (estimate) |
|---|---|
| **Inference on one group of 4–5 images** (Stage 1) | CPU is sufficient. Registration and fusion are dominated by FFTs, warping and sorting 4–5 values per pixel. Use float32 and process tiles if an image exceeds about 4k × 4k on a 16 GB machine. |
| **Inference with learned evidence** (Stages 2–3) | A GPU with 8 GB is comfortable for U-Net or PatchCore inference on tiles; the CPU works but is slower. |
| **Model training** | 8–12 GB VRAM for U-Net on 256–512 px patches with mixed precision. Start with batch 8 at 256 px and adjust from measured peak memory. 16–24 GB allows larger patches or batches and ensembles. |
| **High-resolution / tiled processing** | Tile with overlap (e.g. 512 px tiles, 64 px overlap) and blend; register globally first, then fuse per tile. Keep provenance consistent across tiles. RAM, not VRAM, is the main limit for whole-image fusion. |
| **Optional diffusion / inpainting experiments** | Pretrained model inference: at least the entry GPU, preferably 16–24 GB. Fine-tuning: 24 GB or more. No training from scratch. Treat these as experiments with flagged outputs. |

## H.4 Software stack and reproducible environment

**Core (recommended).**

- Python 3.11 or 3.12
- NumPy, SciPy
- OpenCV (`opencv-python-headless`; SIFT is in the main package since OpenCV 4.4) [46]
- scikit-image (`phase_cross_correlation`, metrics) [47]
- PyTorch + torchvision [48]
- Albumentations (augmentation; installs `opencv-python-headless`, so do not also install `opencv-python`) [49]

- Matplotlib, pandas, PyYAML, tifffile, tqdm, psutil

**Optional.**

- Anomalib (PatchCore, PaDiM) [50]
- segmentation-models-pytorch (U-Net variants)
- timm (backbones)
- faiss-cpu (nearest-neighbour search)
- MLflow or TensorBoard (experiment tracking)
- pytest, pre-commit, ruff (quality)

**Version management.**

- Declare top-level packages with lower bounds in `requirements.in`.
- Compile an exact lock file (`uv pip compile` or `pip-tools`) and commit it.
- Record the PyTorch build (CPU or CUDA variant) and the GPU driver version in every experiment log.
- Optionally build a Docker image from the lock file for the GPU machines.
- Appendix N.3 gives step-by-step commands.

# I. Training and experimentation protocol

## I.1 Data preprocessing and annotation format

**Images.**

- Keep the original bit depth (8- or 16-bit TIFF/PNG) on disk.
- Convert to float32 in [0, 1] at load time, using the dataset's documented maximum (not per-image min–max, which hides contrast changes).
- Crop to the ROI after stage 1; never resize differently across dies of one group.

**Masks.** Single-channel PNG, 0 = background and 255 = defect. A separate "ignore" value (e.g. 128) marks uncertain annotation, excluded from losses.

**Group manifest (JSON).**

```
{
  "group_id": "syn-L03-g0147",
  "split": "test",
  "design_id": "L03",
  "source": "synthetic-v1",
  "images": [{"path": "img_1.tif", "sha256": "…", "die_id": "1",
              "mask": "mask_1.png", "T_true": [[1,0,2.4],[0,1,-1.1]]}],
  "reference": {"path": "R.tif", "kind": "synthetic|reviewed-die|design-render"},
  "generator": {"version": "1.0", "seed": 9182, "params": {"N": 5, "overlap": "all-N:1"}}
}
```

## I.2 Synthetic defect generation and controlled transformations

- **Defect library.** Defect patches are cut from Carinthia-S masks (dilated by 2 px for context) and from MVTec, with source and licence recorded per patch. Procedural defects are bridge, open/break, particle, scratch, residue, missing via and line-edge bump.
- **Insertion.** Random scale (0.5–2×), rotation, intensity matching to the local background, then alpha or Poisson blending. The binary mask is the blended region above a set opacity.
- **Overlap control.** For each group, draw the number of dies sharing each defect location from {1, 2, …, N}. Include all-N cases at a set rate (initial 10% of defect sites) to populate $\boldsymbol{Z}$.
- **Transformations.** Translation, rotation and scale ranges are swept in ablations (Section J.5). Use bicubic resampling with anti-aliasing when scaling down.
- **Imaging variation.** Gain/offset, low-order illumination gradient, Gaussian plus Poisson-like noise, mild defocus blur, and optional SEM-like edge brightening and charging streaks.

- **Augmentation for learned models (training only).** Flips, 90° rotations, brightness/contrast, noise and blur, via Albumentations. No augmentation is applied to validation or test data.

## I.3 Objectives and losses

| Model | Loss | Notes |
|---|---|---|
| U-Net segmentation | BCE + Dice (Equation G.7) | Optional positive-class weight if defects are < 1% of pixels |
| PatchCore / PaDiM | None (fitted, not trained) | Fit on defect-free images of the training split only |
| Masked reconstruction | Heteroscedastic L1 + gradient + SSIM (Equation G.7) | Holes simulated with the same overlap generator |
| Calibration | Temperature scaling or isotonic regression on validation | Never on test [42] |

## I.4 Hyper-parameter starting points (initial values, not optima)

| Parameter | Start | Search range |
|---|---|---|
| U-Net depth / base channels | 4 levels / 32 | 3–5 / 16–64 |
| Patch size | 256 px | 128–512 |
| Batch size | 8 | as memory allows |
| Optimiser | AdamW, lr 1e-3, weight decay 1e-4 | lr 3e-4 – 3e-3 |
| Schedule | cosine decay, 5% warm-up | — |
| Epochs | 100 with early stopping (patience 15) | — |
| PatchCore backbone / layers | WideResNet-50, layers 2+3 (Anomalib default) | ResNet-18 for speed |
| PatchCore coreset ratio | 0.1 | 0.01–0.25 |
| Fusion thresholds $\tau_d, \tau_r$ | 0.5, 0.5 | 0.3–0.7 |
| $\kappa$ (consistency) | 3 | 2–5 |
| $s_{\min}$ | 2 | 1–3 |

## I.5 Checkpoints, seeds, logging and tracking

- Fix and log seeds for Python, NumPy and PyTorch, plus the data-generator seed per group. Enable deterministic cuDNN settings for final runs, noting the slowdown.
- Save checkpoints at each validation improvement and at the end. Store the configuration file, git commit hash, lock-file hash, data-manifest hash, driver and CUDA versions with each run.
- Track with MLflow or TensorBoard: losses, validation metrics, learning rate, peak memory, wall-clock time.
- **Failure-case logging.** For every test group, save a panel (inputs, masks, $\hat{R}$, $U$, $A$, error map). Automatically tag the worst 5% by each primary metric for review.

## I.6 Validation, early stopping and profiling

- Use validation groups (separate designs) for early stopping, threshold tuning and calibration.
- Test once, after freezing all choices; repeat test evaluation only for reported ablations, not for tuning.
- **Profiling run (before any hardware decision).** Run one epoch at the intended patch size and batch on the target machine. Record `torch.cuda.max_memory_allocated()`, peak RSS (psutil) and time per iteration, then scale estimates from these measurements rather than from Section H.

## I.7 Leakage controls

These are the Section D.6 controls enforced in code:

- split by design or acquisition group;
- hash-checked manifests;
- thresholds chosen on validation only;
- no test-time augmentation tuned on test;
- defect patches used for synthetic *test* groups come from a held-out part of the defect library.

# J. Evaluation framework

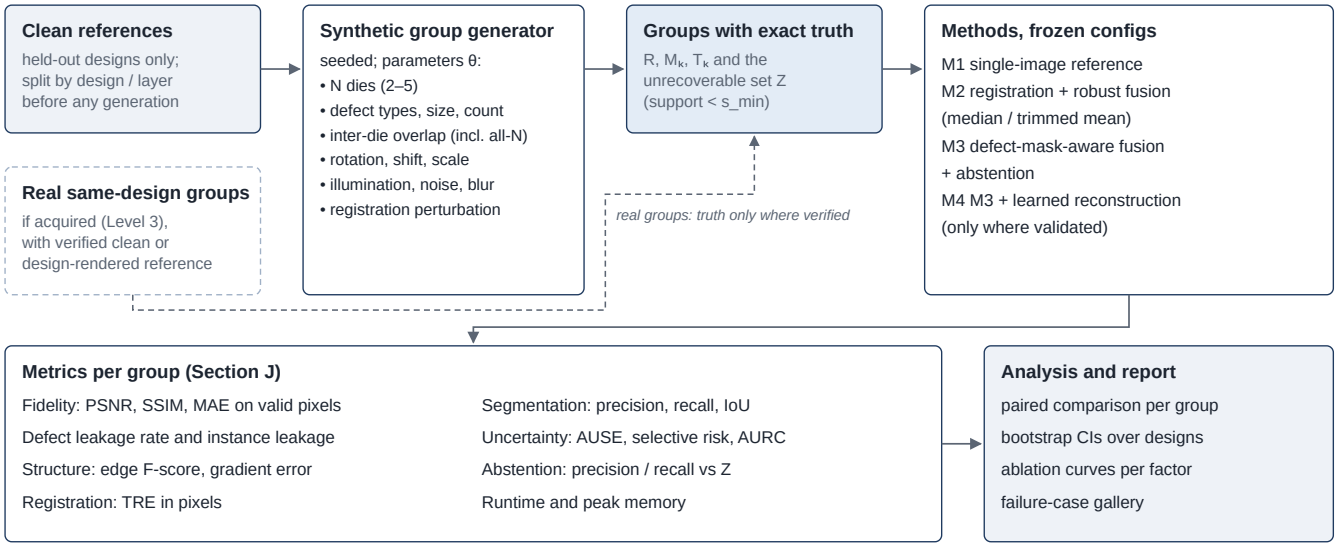



Figure 6. Experimental evaluation pipeline. Schematic of the planned protocol; no results exist yet.

## J.1 Evaluation sets

**Valid pixels.** For a group, let $\Omega_v = \{\mathbf{x} : V_k(\mathbf{x}) = 1 \text{ for all } k\}$ (the common field of view). Unless stated otherwise, fidelity metrics are computed over $\mathcal{E} = \Omega_v \backslash A$, the pixels where the method made an estimate. **Coverage** is reported alongside every metric so that a method cannot look accurate by abstaining everywhere:

$$\text{cov} = \frac{|\Omega_v \backslash A|}{|\Omega_v|}.$$

**Unrecoverable set.** $Z = \{\mathbf{x} \in \Omega_v : \sum_k (1 - M_k(\mathbf{x})) < s_{\min}\}$: pixels without enough clean observations. Computable only where true masks exist (synthetic data, annotated real data).

## J.2 Metrics

**Reconstruction fidelity** (requires a verified clean reference $R$; intensities in [0, 1], so the peak value is $L = 1$):

$$\text{MSE} = \frac{1}{|\mathcal{E}|} \sum_{\mathbf{x} \in \mathcal{E}} \left(\hat{R}(\mathbf{x}) - R(\mathbf{x})\right)^2, \qquad \text{PSNR} = 10 \log_{10} \frac{L^2}{\text{MSE}}, \qquad \text{MAE} = \frac{1}{|\mathcal{E}|} \sum_{\mathbf{x} \in \mathcal{E}} \left|\hat{R} - R\right|.$$

$$\text{SSIM}(u, v) = \frac{(2\mu_u \mu_v + C_1)(2\sigma_{uv} + C_2)}{(\mu_u^2 + \mu_v^2 + C_1)(\sigma_u^2 + \sigma_v^2 + C_2)}, \quad C_1 = (0.01L)^2,\ C_2 = (0.03L)^2,$$

computed in local Gaussian windows and averaged over windows fully inside $\mathcal{E}$ [45]. $\mu$, $\sigma^2$ and $\sigma_{uv}$ are the local means, variances and covariance. **Normalised MAE** (NMAE) divides MAE by the 1st–99th percentile range of $R$, to compare across layouts.

*Limitations:*

- PSNR and MAE reward smooth averages and do not distinguish a blurred edge from a missing feature.
- SSIM depends on window size.
- None of them detects a *plausible but wrong* structure if it is close in intensity.

That is why structure and leakage metrics are primary.

**Defect leakage.** Let $M_\cup = \bigcup_k M_k$ be every pixel that is defective in at least one input. The recoverable, non-abstained defect pixels are $\mathcal{D} = (M_\cup \cap \mathcal{E}) \backslash Z$. With a tolerance $\tau_\ell$ (initial $3\, s_0$):

$$\text{DLR} = \frac{\left|\{\mathbf{x} \in \mathcal{D} : |\hat{R}(\mathbf{x}) - R(\mathbf{x})| > \tau_\ell\}\right|}{|\mathcal{D}|}.$$

**Instance leakage** is the fraction of defect instances (connected components) in which at least $a_{\min}$ pixels leak.

**Silent pass-through** is the most safety-relevant failure: unrecoverable pixels that were *not* abstained:

$$\mathrm{SPR} = \frac{|Z \setminus A|}{|Z|}.$$

*Limitation:* both depend on $\tau_\ell$, so report curves over $\tau_\ell$.

**Structural and edge preservation.** Edge maps $E(\hat{R})$ and $E(R)$ come from the same Canny settings. With matching tolerance $r_t$ (initial 1.5 px):

- $P_e$ = fraction of predicted edge pixels within $r_t$ of a true edge;
- $R_e$ = the converse;
- $F_e = 2P_eR_e/(P_e + R_e)$.

**Gradient error** is $\frac{1}{|\mathcal{E}|}\sum \|\nabla\hat{R} - \nabla R\|_1$. On procedural layouts, also report line-width (critical-dimension proxy) error measured on profiles across lines.

*Limitation:* edge detectors depend on their thresholds, which must be fixed in advance.

**Registration error** (requires known transforms or fiducials). Target registration error, in pixels, over a grid $G$ of points in the ROI:

$$\mathrm{TRE}_k = \frac{1}{|G|}\sum_{\mathbf{x}\in G} \left\|\hat{\mathbf{T}}_k(\mathbf{x}) - \mathbf{T}_k(\mathbf{x})\right\|_2.$$

Report the mean and maximum per image, and the failure rate (TRE above a pre-set limit, e.g. 1 px, or a one-pitch error). On real data, TRE uses manually marked fiducials and is limited by their marking accuracy.

**Defect segmentation** (where true masks exist), from true positives (TP), false positives (FP) and false negatives (FN) at the pixel level:

$$\mathrm{Precision} = \frac{TP}{TP+FP}, \quad \mathrm{Recall} = \frac{TP}{TP+FN}, \quad \mathrm{IoU} = \frac{TP}{TP+FP+FN}.$$

Also report AU-PRO (per-region overlap integrated up to 30% false-positive rate, as defined for MVTec AD [2]) for threshold-free comparison.

*Limitation:* pixel metrics favour large defects; PRO weights each region equally.

**Uncertainty quality.**

- *Sparsification* [43]: sort pixels in $\mathcal{E}$ by $U$, descending. Remove the most uncertain fraction $f$ and compute the MAE of the remainder, $\mathrm{MAE}_U(f)$. The oracle removes pixels by true error, $\mathrm{MAE}_{\mathrm{oracle}}(f)$. Then $\mathrm{AUSE} = \int_0^1 \left(\mathrm{MAE}_U(f) - \mathrm{MAE}_{\mathrm{oracle}}(f)\right) df$; lower is better, and 0 means perfect ranking.
- *Selective risk* [41]: risk at coverage $c$ is the MAE over the fraction $c$ of pixels with the lowest $U$. **AURC** is the area under risk versus coverage.
- *Abstention quality:* the precision and recall of $A$ against $Z$.
- *Calibration of $\hat{p}_k$:* expected calibration error with 15 bins [42].

*Limitation:* $U$ cannot see errors that every reliable die shares; all uncertainty metrics are conditional on the evidence available.

**Runtime and peak memory.**

- Wall-clock time per group: median of 5 runs after one warm-up, with hardware recorded.
- Peak RSS (psutil) and `torch.cuda.max_memory_allocated()`.
- Reported per stage.

## J.3 Methods compared

| ID | Method | Purpose |
|---|---|---|
| M1 | **Single-image reference:** one registered die (random, and best-quality) used directly | Lower bound; mirrors one-neighbour die-to-die practice |
| M2 | **Classical registration + robust fusion:** unweighted median and trimmed mean over all dies | Standard multi-die reference [15] |
| M3 | **Defect-mask-aware fusion:** weights $w_k$, weighted median, abstention | Proposed core (H1–H3) |

| ID | Method | Purpose |
|---|---|---|
| M4 | **M3 + learned reconstruction** for validated hole classes | Tests whether learned fill adds value without hallucination |
| (B) | RASL joint alignment and low-rank fusion [31] | Strong external baseline |

## J.4 Fair-comparison protocol

- All methods receive identical inputs, ROI, registration (unless registration is what is compared) and photometric correction.
- All thresholds are tuned on validation designs and frozen before test evaluation.
- **Coverage-matched comparison.** M3 and M4 abstain and M1 and M2 do not. Report each metric on (i) the pixels where all methods produce estimates, and (ii) each method's own covered pixels, together with coverage. Also report M2 restricted to M3's covered pixels.
- **Primary metrics (pre-registered):** DLR, SPR, and $F_e$ together with PSNR on the common pixels. Other metrics are secondary.
- **Statistics.** The unit of analysis is the *group*, nested in *designs*. Report per-group paired differences, 95% bootstrap confidence intervals (resampling designs, then groups), and a Wilcoxon signed-rank test as secondary. Use Holm correction for multiple comparisons. No method is "better" unless the interval excludes zero; effect sizes are reported either way.

## J.5 Ablation studies

| Factor | Levels (initial plan) | Question |
|---|---|---|
| Number of inputs $N$ | 2, 3, 4, 5 | How fast do leakage and coverage improve with more dies? |
| Defect overlap | independent; pairwise 10/30/50%; all-N 0/10/30% of sites | Where does robust fusion break down, and does abstention catch it? |
| Rotation | 0°, ±0.5°, ±2°, ±5° | Registration capture range |
| Scale | 1, ±1%, ±3% | Magnification drift |
| Illumination | none; gain ±10/25%; gradient; local charging streaks | Normalisation adequacy |
| Injected registration error | 0, 0.25, 0.5, 1, 2 px after registration | H4: sensitivity of fusion to misalignment |
| Missing evidence | random holes; systematic all-N defects | Abstention recall; SPR |
| Evidence source | consistency only; learned only; both | H3 |
| $s_{\min}$ | 1, 2, 3 | Coverage–accuracy trade-off |

## J.6 What cannot be validated without clean references or real same-design data

- **Without a verified clean reference,** PSNR, SSIM, MAE, DLR and edge metrics cannot be computed on real images. Only self-consistency checks remain, such as leave-one-die-out prediction (predict die $k$ from the other dies and compare outside its defects). These indicate agreement, not correctness.
- **Without real same-design groups,** no claim about real-world performance, registration difficulty or synthetic-to-real transfer (H5) can be made. Results remain synthetic.
- **Without design data,** a defect repeated in all dies cannot be distinguished from intended structure, so its recall is unmeasurable on real data.
- **Electrical function** is never validated by this project.

# K. Risk analysis and limitations

| Risk | Effect | Detection | Mitigation |
|---|---|---|---|
| **Repeated defects at the same location** (systematic, mask/reticle-level) | Unrecoverable from images; undetected cases pass silently into $\hat{R}$ | SPR on synthetic all-N cases; on real data only via design data or a golden die | Abstain where learned evidence flags all dies; compare with design rendering or verified golden image where possible [17]; report SPR prominently |
| **Incorrect registration** (incl. one-pitch shifts) | Blurred edges, mixed structures, false defect flags | Residual maps, pitch-ambiguity test, TRE on synthetic | Least flexible model; defect-masked second pass; fiducials or design anchoring; reject images above a residual limit |

| Risk | Effect | Detection | Mitigation |
|---|---|---|---|
| **False-positive defect masks** | Good evidence discarded; lower support; more abstention | Precision on annotated data; coverage drop | Calibrate on validation; require consistency *and* learned evidence for exclusion where support is low |
| **False-negative masks** | Defects leak into $\hat{\boldsymbol{R}}$ | DLR | Noisy-OR combination; dilation; ensembles |
| **Missing circuit structures** (feature absent in all reliable dies due to a shared defect, or occluded) | Reference lacks a real feature; silent if undetected | Edge recall; comparison with design | Same as repeated defects; never "complete" structures from priors without flagging |
| **Texture and illumination variation** (charging, detector differences) | Photometric model fails; spurious deviations | Residual statistics after normalisation | Robust gain/offset; local normalisation only if validated; ablation |
| **False confidence and hallucination** | Plausible but wrong patterns trusted | Hallucination tests (below); AUSE | Abstain by default; learned fill flagged in $\boldsymbol{F}$; never shown as verified |
| **Dataset domain shift** | Learned detectors underperform on target SEM | Performance gap between sources | Fine-tune on target data; report per-source metrics |
| **Synthetic-to-real gap** | Benchmark results do not transfer | Level-3 comparison; realism statistics | Present synthetic results as such; prioritise Level-3 acquisition |
| **Licensing and confidentiality** | Legal or contractual breach; unpublishable results | Licence register per dataset and patch | Respect NC licences (MVTec, KolektorSDD2); NDA for partner data; publish only aggregate metrics or approved images; keep partner data off public repositories |

**Hallucination checks for any learned fill.**

1. *Known-truth test:* on held-out designs, compare the fill with $\boldsymbol{R}$ inside holes. Count *hallucinated edges*: edges in the fill farther than $\boldsymbol{r_t}$ from any true edge, per unit area.
2. *Counterfactual test:* change the hidden content (e.g. remove a via in $\boldsymbol{R}$ under the hole). A trustworthy model should not confidently restore the via.
3. *Context-swap test:* fill the same hole with context from a different design. Similar fills indicate the model is copying priors, not evidence.
4. *Design comparison:* where layout data exists, compare fills with the design rendering.

**Rule.** Uncertain regions stay explicitly marked in every output, figure and report. A plausible but unverified pattern is never shown without its flag.

# L. Implementation roadmap

**Effort assumptions.** One researcher at about 0.8 full-time equivalent, with Python and PyTorch experience, access to the entry-level GPU configuration, and public datasets downloadable at the start. Level-3 data depends on a partner and is *not* on the critical path. Estimates are in person-weeks and are planning figures, not commitments. No calendar dates are given, because the start date and partner timelines are unknown.

| Milestone | Deliverables | Depends on | Acceptance criteria (process-based) | Effort |
|---|---|---|---|---|
| **L1. Literature review and dataset audit** | Systematic review protocol and log (incl. patents); dataset cards with verified licences; refined gap statement | — | Search strings, databases and screening counts documented; every dataset fact linked to the official record; gap statement revised or confirmed | 3–4 |
| **L2. Data preparation and synthetic benchmark** | Generator v1 with seeds; defect library with provenance; split manifests; realism report | L1 | Same seed regenerates identical groups (hash check); splits by design with no hash overlap; realism statistics reported | 3–5 |
| **L3. Registration baseline** | Phase correlation + ECC module; diagnostics; tests | L2 | Recovers synthetic transforms with TRE reported for each test level; unit tests pass; failure cases logged | 2–3 |
| **L4. Defect-mask and fusion baseline** | Cross-die z-score; weights; weighted-median fusion; abstention; M1–M3 results | L3 | Full pipeline runs end to end on test groups; all Section J metrics computed with coverage; deterministic re-run | 3–4 |
| **L5. Learned model experiments** | U-Net, PatchCore/PaDiM evidence; calibration; optional masked U-Net (M4) with hallucination tests | L4 | Trained on train designs only; calibration on validation; H3 ablation complete; M4 enabled only if Section G.6 criteria are met | 4–6 |

| Milestone | Deliverables | Depends on | Acceptance criteria (process-based) | Effort |
|---|---|---|---|---|
| **L6. Evaluation and ablation** | All Section J.5 ablations; statistics; failure gallery; RASL baseline | L4 (L5 for learned rows) | Pre-registered primary metrics reported with CIs; every ablation level has ≥ the planned number of groups | 3–4 |
| **L7. Final report and demonstration** | Paper/report, slides, demo notebook, released code and benchmark (where licences allow) | L6 | Results reproducible from the repository by a second person; limitations section complete | 2–3 |
| (Parallel) **Level-3 data acquisition** | Data agreement; real groups; annotations | Partner | Annotation QC (Section D.5) passed | Partner-dependent |

**Total for L1–L7:** about 20–29 person-weeks, plus partner-dependent real-data work and writing time for any publication.

# M. Expected outputs

| Output | Content | Form |
|---|---|---|
| Research proposal | One-page faculty proposal and this document | PDF, DOCX, Markdown sources |
| Dataset audit | Dataset cards, licence register, suitability matrix (Figure 1), verification log | Markdown + CSV |
| Reproducible code repository | Modules for stages 1–9; configs; tests; lock file; Dockerfile; generator | Git repository |
| Trained checkpoints (if feasible) | U-Net, PatchCore memory banks, optional reconstruction model, each with training config and data-manifest hash | Files + model cards |
| Evaluation scripts | Metric implementations (Section J) with unit tests on toy cases with known answers | Python package + CLI |
| Qualitative and quantitative results | Metric tables with CIs; ablation curves; failure gallery, synthetic and real reported separately | Report figures, CSV |
| Limitations and future work | What was not validated (Section J.6); open questions (one-page summary) | Report section |
| Final paper/report and presentation | Written report; slides; optional short animation (Appendix N.6) | PDF, PPTX |

**Not an output:** any claim of physical die repair, electrical functionality, or real-world accuracy without Level-3 data.

# N. References and appendices

## References


**Datasets and dataset papers**

[1] P. Bergmann, M. Fauser, D. Sattlegger, C. Steger, "MVTec AD — A comprehensive real-world dataset for unsupervised anomaly detection," *CVPR*, 2019, pp. 9584–9592. https://doi.org/10.1109/CVPR.2019.00982 · dataset: https://www.mvtec.com/research-teaching/datasets/mvtec-ad

[2] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, 129(4):1038–1059, 2021. https://doi.org/10.1007/s11263-020-01400-4

[3] L. Heckler-Kram, J.-H. Neudeck, U. Scheler, R. König, C. Steger, "The MVTec AD 2 dataset: Advanced scenarios for unsupervised anomaly detection," *Int. J. Comput. Vis.*, 2026. https://doi.org/10.1007/s11263-026-02743-0 · https://arxiv.org/abs/2503.21622 · dataset: https://www.mvtec.com/research-teaching/datasets/mvtec-ad-2

[4] C. Kofler, S. Strauß, A. Zernig *et al.*, "Carinthia dataset," v2, Zenodo, 2024. https://zenodo.org/records/10715190

[5] C. Kofler, V. Hasić, "Carinthia-S," Zenodo, 2025. https://zenodo.org/records/16895427

[6] J. Božič, D. Tabernik, D. Skočaj, "Mixed supervision for surface-defect detection: From weakly to fully supervised learning," *Computers in Industry*, 2021. https://doi.org/10.1016/j.compind.2021.103459 · https://arxiv.org/abs/2104.06064 · dataset (KolektorSDD2): https://www.vicos.si/resources/kolektorsdd2/

[7] Y. Zou, J. Jeong, L. Pemula, D. Zhang, O. Dabeer, "SPot-the-difference self-supervised pre-training for anomaly detection and segmentation," *ECCV*, 2022 (VisA dataset). https://arxiv.org/abs/2207.14315 · https://github.com/amazon-science/spot-diff

[8] M.-J. Wu, J.-S. R. Jang, J.-L. Chen, "Wafer map failure pattern recognition and similarity ranking for large-scale data sets," *IEEE Trans. Semicond. Manuf.*, 28(1):1–12, 2015 (WM-811K). https://doi.org/10.1109/TSM.2014.2364237

[9] "Mixed-type wafer defect datasets" (MixedWM38), IEEE DataPort. https://ieee-dataport.org/node/15177

**Semiconductor inspection**

[10] K. Taha, "Observational and experimental insights into machine learning-based defect classification in wafers," *J. Intell. Manuf.*, Springer. https://doi.org/10.1007/s10845-024-02521-0

[11] E. Dehaerne, B. Dey, V. Blanco, J. Davis, "Scanning electron microscopy-based automatic defect inspection for semiconductor manufacturing: A systematic review," arXiv:2409.06833 (accepted, *J. Micro/Nanopatterning, Materials, and Metrology*). https://arxiv.org/abs/2409.06833

[12] B. Dey, E. Dehaerne, K. Khalil, S. Halder, P. Leray, M. A. Bayoumi, "Deep learning based defect classification and detection in SEM images: A Mask R-CNN approach," arXiv:2211.02185, 2022. https://arxiv.org/abs/2211.02185

[13] V. De Ridder, B. Dey, S. Halder, B. Van Waeyenberge, "SEMI-DiffusionInst: A diffusion model based approach for semiconductor defect classification and segmentation," arXiv:2307.08693, 2023. https://arxiv.org/abs/2307.08693

[14] M. Zontak, I. Cohen, "Defect detection in patterned wafers using anisotropic kernels," *Machine Vision and Applications*, 21:129–141, 2010. https://doi.org/10.1007/s00138-008-0146-y

[15] KLA-Tencor, "Methods for generating a standard reference die for use in a die to standard reference die inspection and methods for inspecting a wafer," US 8,204,296 (publ. US 2009/0041332 A1). https://patents.google.com/patent/US20090041332A1/en

[16] KLA-Tencor, "Wafer inspection using difference images," US 11,270,430 (publ. US 2018/0342051 A1). https://patents.google.com/patent/US20180342051A1/en

[17] KLA-Tencor, "Detecting defects on a wafer using defect-specific information," US 9,189,844. https://patents.google.com/patent/US9189844

**Anomaly detection and segmentation**

[18] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, P. Gehler, "Towards total recall in industrial anomaly detection," *CVPR*, 2022, pp. 14318–14328. https://doi.org/10.1109/CVPR52688.2022.01392

[19] T. Defard, A. Setkov, A. Loesch, R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," ICPR 2020 Workshops, 2021. https://arxiv.org/abs/2011.08785

[20] V. Zavrtanik, M. Kristan, D. Skočaj, "DRAEM — A discriminatively trained reconstruction embedding for surface anomaly detection," *ICCV*, 2021. https://arxiv.org/abs/2108.07610

[21] S. Akcay, D. Ameln, A. Vaidya, B. Lakshmanan, N. Ahuja, U. Genc, "Anomalib: A deep learning library for anomaly detection," arXiv:2202.08341, 2022. https://arxiv.org/abs/2202.08341

[22] O. Ronneberger, P. Fischer, T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” *MICCAI*, 2015. https://doi.org/10.1007/978-3-319-24574-4_28 · https://arxiv.org/abs/1505.04597

**Registration**

[23] B. Zitová, J. Flusser, “Image registration methods: A survey,” *Image and Vision Computing*, 21(11):977–1000, 2003. https://doi.org/10.1016/S0262-8856(03)00137-9

[24] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *Int. J. Comput. Vis.*, 60(2):91–110, 2004. https://doi.org/10.1023/B:VISI.0000029664.99615.94

[25] E. Rublee, V. Rabaud, K. Konolige, G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” *ICCV*, 2011. https://doi.org/10.1109/ICCV.2011.6126544

[26] M. A. Fischler, R. C. Bolles, “Random sample consensus,” *Commun. ACM*, 24(6):381–395, 1981. https://doi.org/10.1145/358669.358692

[27] G. D. Evangelidis, E. Z. Psarakis, “Parametric image alignment using enhanced correlation coefficient maximization,” *IEEE TPAMI*, 30(10):1858–1865, 2008. https://doi.org/10.1109/TPAMI.2008.113

[28] B. S. Reddy, B. N. Chatterji, “An FFT-based technique for translation, rotation, and scale-invariant image registration,” *IEEE Trans. Image Process.*, 5(8):1266–1271, 1996. https://doi.org/10.1109/83.506761

[29] M. Guizar-Sicairos, S. T. Thurman, J. R. Fienup, “Efficient subpixel image registration algorithms,” *Optics Letters*, 33(2):156–158, 2008. https://doi.org/10.1364/OL.33.000156

**Robust fusion**

[30] E. J. Candès, X. Li, Y. Ma, J. Wright, “Robust principal component analysis?” *J. ACM*, 58(3):11, 2011. https://doi.org/10.1145/1970392.1970395

[31] Y. Peng, A. Ganesh, J. Wright, W. Xu, Y. Ma, “RASL: Robust alignment by sparse and low-rank decomposition for linearly correlated images,” *IEEE TPAMI*, 34(11):2233–2246, 2012. https://doi.org/10.1109/TPAMI.2011.282

[32] S. Farsiu, M. D. Robinson, M. Elad, P. Milanfar, “Fast and robust multiframe super resolution,” *IEEE Trans. Image Process.*, 13(10):1327–1344, 2004. Author PDF: https://people.duke.edu/~sf59/SRfinal.pdf

**Inpainting and masked reconstruction**

[33] A. Telea, “An image inpainting technique based on the fast marching method,” *J. Graphics Tools*, 9(1):23–34, 2004. https://doi.org/10.1080/10867651.2004.10487596

[34] G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao, B. Catanzaro, “Image inpainting for irregular holes using partial convolutions,” *ECCV*, 2018. https://arxiv.org/abs/1804.07723

[35] R. Suvorov *et al.*, “Resolution-robust large mask inpainting with Fourier convolutions,” *WACV*, 2022, pp. 2149–2159. https://openaccess.thecvf.com/content/WACV2022/html/Suvorov_Resolution-Robust_Large_Mask_Inpainting_With_Fourier_Convolutions_WACV_2022_paper.html

[36] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, L. Van Gool, “RePaint: Inpainting using denoising diffusion probabilistic models,” *CVPR*, 2022, pp. 11461–11471. https://doi.org/10.1109/CVPR52688.2022.01117

[37] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick, “Masked autoencoders are scalable vision learners,” *CVPR*, 2022. https://arxiv.org/abs/2111.06377

**Uncertainty and hallucination**

[38] A. Kendall, Y. Gal, “What uncertainties do we need in Bayesian deep learning for computer vision?” *NeurIPS*, 2017. https://arxiv.org/abs/1703.04977

[39] B. Lakshminarayanan, A. Pritzel, C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” *NeurIPS*, 2017. https://arxiv.org/abs/1612.01474

[40] Y. Gal, Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” *ICML*, 2016. https://arxiv.org/abs/1506.02142

[41] Y. Geifman, R. El-Yaniv, “Selective classification for deep neural networks,” *NeurIPS*, 2017. https://arxiv.org/abs/1705.08500

[42] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, “On calibration of modern neural networks,” *ICML*, 2017. https://arxiv.org/abs/1706.04599

[43] E. Ilg *et al.*, “Uncertainty estimates and multi-hypotheses networks for optical flow,” *ECCV*, 2018, pp. 652–667. https://doi.org/10.1007/978-3-030-01234-2_40

[44] S. Bhadra, V. A. Kelkar, F. J. Brooks, M. A. Anastasio, “On hallucinations in tomographic image reconstruction,” *IEEE Trans. Med. Imaging*, 40(11):3249–3260, 2021. https://doi.org/10.1109/TMI.2021.3077857

**Metrics and software**

[45] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, 13(4):600–612, 2004. https://doi.org/10.1109/TIP.2003.819861

[46] OpenCV documentation. https://docs.opencv.org/4.x/

[47] scikit-image documentation. https://scikit-image.org/docs/stable/

[48] PyTorch: installation selector and documentation. https://pytorch.org/get-started/locally/

[49] Albumentations documentation. https://albumentations.ai/docs/

[50] Anomalib documentation. https://anomalib.readthedocs.io/

## N.1 Verification log

- **Checked via search results quoting the official or publisher page** (direct access was blocked in the build environment):
  - references [1]–[8], [10]–[14], [15]–[18], [19]–[21], [23], [27]–[31], [33]–[36], [43], [44];
  - dataset facts in Section D.2.
- **Cited from established bibliographic knowledge, not re-fetched:** author list of [7], [22], [24]–[26], [37]–[42], [45]–[50].
- **Not confirmed:**
  - Carinthia licence;
  - Carinthia record identity (concept vs. version DOI);
  - Carinthia-S licence, resolution and class semantics (third-party only);
  - VisA licence (conflicting sources);
  - WM-811K and MixedWM38 licences;
  - MixedWM38 associated paper and DOI;
  - Farsiu et al. DOI;
  - DOIs of PaDiM, DRAEM and LaMa proceedings versions (arXiv/CVF links used instead);
  - publication year of [10].
- **Action:** open every link once by hand before external distribution.

## N.2 Dataset cards (summary)

| Field | Carinthia | Carinthia-S | MVTec AD | MVTec AD 2 | KolektorSDD2 | VisA |
|---|---|---|---|---|---|---|
| Modality | SEM, grayscale | SEM, grayscale | RGB/gray photos | RGB/gray photos | Gray photos | RGB photos |
| Size | 4,591 images | 4,591 images + masks | 5,354 | > 8,000 | 3,335 | 10,821 |
| Labels | Image class (6) | Binary masks + class | Pixel masks | Pixel masks (public test) | Pixel masks | Pixel masks |
| Defect-free set | Class 6 (no visible defect; third-party) | Same | Training set | Training set | 2,979 images | 9,621 normal |
| Patterned die / same-design groups | No / No | No / No | No / No | No / No | No / No | PCB proxy / partial |
| Licence | Not verified | CC BY 4.0 (third-party) | CC BY-NC-SA 4.0 | CC BY-NC-SA 4.0 | CC BY-NC-SA 4.0 | Conflicting |
| Use here | SEM appearance; pre-training | Masks; U-Net; defect library | AD baselines; defect library | Lighting robustness | Weak/mixed supervision | Registration proxy |

## N.3 Installation

```
        # 1. Python environment (uv shown; conda/mamba is equivalent)
        uv venv --python 3.11 .venv
        source .venv/bin/activate

        # 2. PyTorch: pick the CPU or CUDA build from https://pytorch.org/get-started/locally/
        #    and install it with the index URL the selector prints, e.g.
        #    uv pip install torch torchvision --index-url <URL from selector>

        # 3. Everything else, then freeze exact versions
        uv pip install -r requirements.in
        uv pip freeze > requirements.lock     # or: uv pip compile requirements.in -o requirements.lock

        # 4. Sanity checks
        python -c "import torch, cv2, skimage; print(torch.__version__, torch.cuda.is_available(),
cv2.__version__, skimage.__version__)"
        pytest -q
```

`requirements.in` (lower bounds only; the lock file pins exact versions):

```
numpy>=1.26
scipy>=1.11
opencv-python-headless>=4.8
scikit-image>=0.22
albumentations>=1.4
matplotlib>=3.8
pandas>=2.1
pyyaml>=6.0
tifffile>=2023.9
tqdm>=4.66
psutil>=5.9
mlflow>=2.9           # or tensorboard
pytest>=7.4
# optional
anomalib
segmentation-models-pytorch
timm
faiss-cpu
```

## N.4 Configuration example

```yaml
experiment: m3_weighted_median_v1
seed: 1234
data:
  manifest: data/manifests/synthetic_v1_test.json
  roi: auto
registration:
  model: euclidean            # translation | euclidean | affine
  phase_upsample: 20
  ecc: {iterations: 200, eps: 1.0e-6, defect_masked_second_pass: true}
  residual_window: 15
  sigma_rho: 0.2
  pitch_ambiguity_check: true
normalisation: {method: robust_affine, trim_fraction: 0.2}
evidence:
  consistency: {kappa: 3.0, smoothing_sigma: 0.7, noise_floor: auto}
  learned: {kind: none}       # none | unet | patchcore | padim
  calibration: temperature
  mask_postprocess: {open_px: 1, dilate_px: 2}
reliability: {tau_d: 0.5, tau_r: 0.5}
fusion:
  estimator: weighted_median  # weighted_median | huber | weighted_mean
  structure_check: {region_px: 24, tau_e: auto}
abstention: {s_min: 2, tau_u_noise_units: 2.0}
reconstruction: {enabled: false}
evaluation:
  leakage_tau_noise_units: 3.0
  edge_tolerance_px: 1.5
  bootstrap: {resamples: 2000, unit: design}
export: {reference_dtype: float32, nan_for_abstained: true, save_provenance: true}
```

## N.5 Glossary

| Term | Meaning |
|---|---|
| Abstention | Declining to output a value at a pixel because evidence is insufficient; recorded in mask $\boldsymbol{A}$. |
| AUSE | Area under the sparsification error curve: how well uncertainty ranks errors. |
| AU-PRO | Area under the per-region-overlap curve, a threshold-free segmentation metric. |
| Breakdown point | Fraction of outliers an estimator tolerates before it can be arbitrarily wrong (median: 50%). |
| Die-to-die inspection | Comparing a die image with images of neighbouring dies of the same design. |
| ECC | Enhanced correlation coefficient registration [27]. |
| Golden die / reference | A die image verified to be defect-free, used as reference. |
| Hallucination | Plausible structure produced by a model that is not supported by the measurements. |
| MAD | Median absolute deviation, a robust spread estimate. |
| Provenance map | Per-pixel record of which dies contributed to the reconstructed value. |
| ROI | Region of interest common to all images. |
| SEM | Scanning electron microscope. |
| Support $\boldsymbol{S(\mathbf{x})}$ | Number of reliable observations at a pixel. |
| Systematic defect | A defect repeated at the same location on many or all dies (e.g. from the mask or design). |
| TRE | Target registration error: distance between estimated and true mapped points. |
| Wafer map | Grid of per-die categorical results; not an image of the die surface. |

## N.6 Optional animation storyboard (about 60 seconds)

The written document is complete without this animation. All frames would be synthetic and labelled “Conceptual example — not experimental data”.

| Scene | Duration | Visual | Narration / caption |
|---|---|---|---|
| 1 | 6 s | One layout; five dies appear, rotated and shifted, each with a red-ringed defect | “Five images of the same design, each with its own defect.” |

| Scene | Duration | Visual | Narration / caption |
|---|---|---|---|
| 2 | 8 s | Images rotate into a common grid; crosshair overlay | “Registration aligns every image to one frame.” |
| 3 | 8 s | Each image fades to its estimated defect map (red = defective, hatched = uncertain) | “Estimated maps mark which pixels to distrust; they are estimates, not labels.” |
| 4 | 12 s | For region a, ticks light up on dies 2–5; their pixels flow into the output | “Each region is rebuilt only from dies that are reliable there.” |
| 5 | 10 s | Region f: all five dies show ×; the output region becomes hatched | “Where every die is defective, nothing is invented; the region is flagged.” |
| 6 | 8 s | Side-by-side reference and uncertainty map | “Output: a candidate reference plus an uncertainty map.” |
| 7 | 8 s | Text card | “A digital reference for inspection, not a repaired die, and not evidence of electrical function.” |